

UNIVERSITÀ DEGLI STUDI ROMA TRE

Dipartimento di Ingegneria Civile, Informatica
e delle Tecnologie Aeronautiche
Corso di Laurea Ingegneria Informatica e dell'Intelligenza Artificiale

Tesi di Laurea

Agent-based Earth Observation discovery: un approccio multi-agentico per l'accesso al dato nei cataloghi di osservazione della terra

Laureando

Alessio Pecilli

Matricola 589232

Relatore

Prof. Matteo Rosati

Anno Accademico 2025/2026

Indice

Indice **ii**

1 Introduzione **1**

1.1 Posizionamento rispetto ai lavori correlati 2

1.2 Il problema affrontato 2

1.3 Obiettivo del lavoro 3

1.4 Impostazione del progetto 5

1.5 Perimetro e limiti 6

2 Dall'intelligenza artificiale agli agenti basati su LLM **7**

2.1 Intelligenza artificiale e nozione di agente 7

2.2 Apprendimento statistico e reti profonde 8

2.2.1 Dai sistemi simbolici all'apprendimento dai dati 8

2.2.2 Machine learning 8

2.2.3 Deep learning 9

2.3 Modelli linguistici e architettura Transformer 10

2.3.1 Modellazione del linguaggio 10

2.3.2 Transformer e attenzione 11

2.3.3 Modelli linguistici di grandi dimensioni 11

2.3.4 Pre-addestramento, adattamento e limiti degli LLM isolati . . . 12

2.4 Agenti basati su LLM 12

3 Model Context Protocol: contesto, strumenti e interoperabilità **14**

3.1 Problema e scelta architetturale 14

3.2 Concetti principali . . . 15
3.2.1 Host . . . 15
3.2.2 Client . . . 15
3.2.3 Server . . . 15
3.2.4 Trasporto e messaggi . . . 16
3.3 Capacità esposte dai server . . . 16
3.3.1 Tools . . . 16
3.3.2 Capacità non utilizzate nel prototipo . . . 16
3.4 Uso di MCP nel workflow . . . 17
3.5 Confronto e limiti . . . 17
3.5.1 Integrazione diretta . . . 17
3.5.2 Tool calling . . . 18
3.5.3 Costi introdotti . . . 18
3.6 Sicurezza e gestione operativa . . . 18
3.6.1 Autenticazione e autorizzazione . . . 18
3.6.2 Prompt injection e dati non fidati . . . 18
3.6.3 Osservabilità . . . 19
3.7 Criteri adottati nel progetto . . . 19

4 Fonti informative geospaziali: GDACS e Copernicus 20
4.1 Componenti applicativi . . . 20
4.1.1 Interfaccia e API . . . 20
4.1.2 Orchestrazione e memoria . . . 21
4.1.3 Modello e distribuzione . . . 22
4.2 GDACS . . . 24
4.2.1 Scopo e contesto operativo . . . 24
4.2.2 Eventi e livelli di allerta . . . 24
4.2.3 Accesso programmatico . . . 25
4.2.4 Limiti . . . 25
4.3 Il programma Copernicus . . . 25
4.4 Copernicus Data Space Ecosystem e STAC . . . 26
4.4.1 Ricerca delle acquisizioni . . . 26

4.4.2 Dal metadato alla heatmap 26
4.5 Copernicus Emergency Management Service 27
4.6 Marine e Drought nel catalogo di Discovery 28
4.6.1 Metadati Marine 28
4.6.2 Metadati Drought 28
4.6.3 Due usi distinti di STAC 28
4.7 Confronto e complementarità 29
4.8 Implicazioni per la progettazione dell'agente 29

5 Architettura del sistema proposto 31
5.1 Obiettivo e requisiti 31
5.2 Vista architetturale 32
5.3 Organizzazione del backend e del frontend 33
5.4 Il grafo degli agenti 35
5.4.1 Percorso principale 35
5.4.2 Gateway e confine di dominio 36
5.4.3 Parsing dell'intento 36
5.4.4 Orchestratore e piano di esecuzione 36
5.4.5 Il ramo Discovery 37
5.5 Catalogo a grafo e pipeline di harvesting 38
5.5.1 Modello del dominio 38
5.5.2 Raccolta, aggiornamento e riconciliazione 38
5.5.3 Infrastruttura e confine di lettura 39
5.6 Agenti specializzati e nodi di elaborazione 40
5.7 Contratti e stato condiviso 41
5.8 Persistenza 41
5.8.1 Checkpoint conversazionali 41
5.8.2 Archivio degli artefatti 42
5.9 Dashboard generativa e verificabile 42
5.9.1 Leggibilità e correttezza dei grafici 43
5.10 Integrazione MCP 43
5.11 Distribuzione tramite container 44

5.12 Sicurezza e osservabilità 44

6 Funzionamento end-to-end, casi d'uso e verifica 47

6.1 Ingresso della richiesta 47
6.2 Scenario 1: analisi multi-sorgente di un pericolo 48
6.2.1 Risoluzione semantica 48
6.2.2 Pianificazione ed esecuzione 49
6.2.3 Sincronizzazione e risposta 50
6.3 Scenario 2: interrogazione di una attivazione CEMS 50
6.4 Scenario 3: follow-up e riuso della memoria 51
6.5 Scenario 4: scoprire un dataset nel grafo 52
6.6 Dashboard Copernicus 53
6.6.1 Dati, provenienza e composizione 53
6.6.2 Grafici leggibili e senza sovrapposizioni 54
6.6.3 Interazione, revisioni e chat contestuale 55
6.6.4 Heatmap Copernicus 55
6.7 Streaming della risposta 55
6.8 Gestione degli errori 56
6.8.1 Errore di una sorgente 56
6.8.2 Timeout e retry 57
6.8.3 Errore della memoria 57
6.9 Matrice dei percorsi e delle evidenze 57
6.10 Strategia di verifica 57
6.10.1 Test unitari 59
6.10.2 Test del workflow 59
6.10.3 Test delle integrazioni 59
6.11 Limiti attuali 60

7 Osservabilità del workflow e diagnosi 61

7.1 Log, metriche e trace 61
7.2 Architettura della telemetria 62
7.3 Strumentazione del backend 63

7.3.1 Bootstrap e wrapper 63
7.3.2 HTTP, streaming e correlazione 64
7.3.3 Workflow, nodi e dipendenze 64
7.3.4 Invocazioni LLM, attesa e token 65
7.4 Interpretazione delle metriche 66
7.4.1 Contatori, gauge e istogrammi 66
7.4.2 Popolazioni e denominatori 66
7.5 Dashboard Grafana e alert 67
7.6 Diagnosi e limiti della copertura 68

8 Valutazione con LLM-as-a-Judge 70
8.1 Oggetto e perimetro della valutazione 70
8.2 Dataset ed evidenze di riferimento 72
8.2.1 Composizione dei casi 72
8.2.2 Costruzione della ground truth 72
8.3 Esecuzione del vero workflow 73
8.4 Controlli deterministici e rubriche ordinali 73
8.4.1 Controlli sui fatti osservati 73
8.4.2 Dimensioni e soglie 74
8.4.3 Prompt, parsing e coerenza 75
8.5 Stati finali e denominatori 75
8.6 Incertezza e misure di recupero 76
8.6.1 Intervallo di Wilson 76
8.6.2 Bootstrap per famiglie e voti ordinali 76
8.6.3 Indicatori indipendenti dai voti 76
8.7 Confronti e ripetizioni 77
8.7.1 Riproducibilità e stabilità 77
8.7.2 Confronti appaiati fra run 77
8.8 Risultati documentati 77
8.8.1 Run complete con protocollo 6.0 77
8.8.2 Confronto storico con protocollo 4.0 79
8.9 Interpretazione e minacce alla validità 79

9 Conclusioni e sviluppi futuri **80**

Bibliografia **82**

Capitolo 1

Introduzione

Una domanda relativa a un'emergenza naturale raramente trova risposta in una sola fonte. Sapere se si è verificato un terremoto, verificare quali acquisizioni satellitari coprano la zona e controllare se sia stata attivata una produzione cartografica sono operazioni diverse. I dati possono riguardare lo stesso luogo e lo stesso intervallo temporale, ma descrivono fenomeni distinti e hanno tempi di aggiornamento, formati e livelli di affidabilità propri.

Si consideri, ad esempio, una richiesta sui terremoti avvenuti in Italia nell'ultima settimana e sulla relativa copertura satellitare. La prima parte richiede un elenco di eventi e dei rispettivi livelli di allerta; la seconda riguarda la disponibilità di osservazioni della Terra, non la misura dei danni. Un'eventuale attivazione di un servizio di emergenza aggiunge poi informazioni su aree di interesse e prodotti cartografici. Riunire questi elementi in una sola risposta è utile soltanto se la loro provenienza rimane riconoscibile.

Questa tesi affronta il problema realizzando *Copernicus Data Workspace*: uno spazio di lavoro conversazionale che coordina fonti geospaziali indipendenti. L'utente formula la richiesta in linguaggio naturale; il sistema ne ricava luogo, periodo e obiettivo, sceglie le fonti pertinenti e restituisce testo, dati strutturati e visualizzazioni. Il modello linguistico aiuta a interpretare la domanda e a formulare la risposta, ma non viene trattato come archivio degli eventi né come origine dei valori mostrati nei grafici.

1.1 Posizionamento rispetto ai lavori correlati

Il progetto si colloca nella linea degli agenti basati su LLM che decidono quando invocare strumenti esterni [35, 30, 23]. Rispetto a un assistente generico con function calling, il perimetro è ristretto all'osservazione della Terra e alle fonti di allerta: catalogo metadati Copernicus, STAC live, GDACS e CEMS. Sistemi commerciali o sperimentali di "Earth Copilot" o chat su cataloghi EO condividono l'idea di interrogare metadati in linguaggio naturale; qui si insiste sulla separazione fra Discovery (risorsa catalogata) e acquisizioni STAC live, sul ranking deterministico e su un protocollo di valutazione con controlli critici. MCP [19] è usato come contratto di integrazione per GDACS e CEMS, non come soluzione alla semantica geospaziale. Il contributo non è un nuovo algoritmo di retrieval, bensì l'organizzazione di questi elementi in un workspace verificabile.

1.2 Il problema affrontato

Le applicazioni conversazionali rendono semplice l'accesso a servizi che, presi singolarmente, richiederebbero la conoscenza di endpoint, parametri e formati di risposta. La semplificazione dell'interfaccia non elimina però la complessità dei dati sottostanti. Nel percorso operativo sono coinvolte tre famiglie di informazioni, alle quali si affianca un catalogo trasversale di metadati.

GDACS pubblica eventi e allerte relativi a disastri naturali. Il catalogo del Copernicus Data Space Ecosystem descrive invece le acquisizioni satellitari disponibili per una certa area e un certo periodo. Copernicus Emergency Management Service documenta attivazioni e prodotti cartografici predisposti a supporto della gestione delle emergenze. Queste fonti possono essere consultate nello stesso turno, ma non devono essere fuse come se misurassero la stessa grandezza. Il numero di acquisizioni satellitari, per esempio, non esprime la gravità di un evento.

Alla distinzione semantica si aggiunge quella tecnica. GDACS distribuisce anche feed XML, il catalogo satellitare espone un'interfaccia STAC e CEMS adotta strutture specifiche per le proprie attivazioni. I servizi possono restituire risposte vuote, diventare temporaneamente indisponibili o richiedere trattamenti diversi degli errori. Collocare

questi dettagli nei prompt renderebbe difficile capire se un'informazione sia stata recuperata oppure generata dal modello.

Anche l'interpretazione linguistica deve essere verificata. Un LLM sa trattare espressioni come "la settimana scorsa", riconoscere un codice CEMS e comporre un testo leggibile, ma la sua conoscenza può essere incompleta o non aggiornata. La plausibilità di una frase non dimostra che il dato corrispondente esista nella fonte consultata. Nel progetto interpretazione, accesso ai servizi e calcoli sono perciò passaggi separati.

I turni successivi richiedono inoltre di conservare il contesto. Dopo una tabella o un grafico, l'utente può chiedere di mostrare una sola piattaforma, cambiare l'aggregazione o aggiornare una sorgente. Il significato di questi comandi dipende dai risultati precedenti; lo stato deve quindi essere associato alla conversazione senza inserire ogni dato remoto nell'intera cronologia testuale.

La verificabilità riguarda sia il testo sia le visualizzazioni. Link, identificativi, timestamp e criteri di aggregazione fanno parte del risultato. Un grafico deve indicare i record e i campi dai quali deriva; il modello può proporre la vista, mentre conteggi e misure vengono calcolati dal software.

1.3 Obiettivo del lavoro

L'obiettivo è un prototipo che trasformi una richiesta geografica in un percorso di consultazione controllato. Il sistema deve poter interrogare più fonti quando la domanda lo richiede, continuare a produrre una risposta se una sorgente secondaria non è disponibile e mantenere il contesto necessario per i turni successivi. I risultati devono restare associati alla fonte dalla quale provengono.

La tesi non addestra un nuovo modello linguistico. Studia l'architettura che lo circonda: ammissione della domanda nel dominio, risoluzione dei parametri, scelta delle sorgenti e restituzione dei risultati nel workspace. Le ambiguità linguistiche sono affidate al modello; schemi, filtri, calcoli e persistenza rimangono sotto il controllo del codice.

Il contributo dichiarato è **progettuale e implementativo**: un prototipo con responsabilità specializzate, percorsi Discovery/operativo controllati e artefatti verificabili. Il titolo parla di approccio multiagentico perché il workflow suddivide interpretazione, recupero e composizione fra nodi distinti; questa organizzazione non è però confrontata,

in questa tesi, con un agente singolo o con una pipeline più semplice nelle stesse condizioni. Non si sostiene quindi un vantaggio quantitativo di accuratezza, latenza o costo rispetto a tali alternative: tale affermazione richiederebbe una baseline controllata.

Il percorso **operativo** combina la ricerca web con tre fonti strutturate. GDACS fornisce eventi e allerte; il catalogo STAC di Copernicus descrive acquisizioni satellitari e relativa copertura; CEMS permette di consultare attivazioni di Rapid Mapping e Risk and Recovery Mapping. Il contenuto della domanda determina quali percorsi siano necessari e più rami possono essere attivati nello stesso turno.

Il percorso di **Discovery** affronta una domanda complementare: quali risorse Copernicus permettono di studiare una certa variabile, area o periodo? Tre harvester raccolgono metadati Marine, CDSE/STAC e Drought in NebulaGraph. Il workflow consulta questo grafo per individuare dataset, variabili, collezioni e layer pertinenti. La scoperta di un dataset di temperatura non equivale al download dei suoi valori; analogamente, il catalogo locale non sostituisce la ricerca delle acquisizioni sul servizio STAC remoto. Sono ammessi anche percorsi **combinati**, in cui Discovery e analisi operativa convivono nella stessa sessione senza confondere metadati di catalogo e scene satellitari live.

Per GDACS e CEMS è stato adottato il Model Context Protocol. I dettagli dei due servizi vengono incapsulati in server separati, ai quali il backend accede attraverso un client comune. Il catalogo STAC live rimane integrato direttamente nell'applicazione. La presenza dei due approcci nello stesso progetto permette di confrontarne gli effetti sull'organizzazione del codice, senza assumere che un protocollo risolva automaticamente ogni problema di integrazione.

Il punto di ingresso è la dashboard nativa autenticata con Keycloak (`http://localhost:8080/dashboard`): home in italiano con selettore IT/EN, formulazione della domanda, suggerimenti e ripresa delle indagini recenti. La risposta operativa può includere una dashboard delle acquisizioni Copernicus. Grafici, indicatori, tabella e mappa derivano dai record STAC normalizzati; GDACS, CEMS e ricerca web restano nella parte testuale o in sezioni dedicate, perché le rispettive quantità non sono confrontabili nelle stesse serie. Durante la conversazione l'utente può applicare filtri, cambiare vista o chiedere una nuova aggregazione. Ogni modifica produce una revisione dell'artefatto,

ma i valori continuano a essere calcolati sul dataset conservato.

La Discovery restituisce invece un artefatto di risorse catalogate con identificativi, descrizioni e collegamenti. Sono quindi distinte la vista dei metadati e quella delle acquisizioni. A entrambe si affiancano un'infrastruttura di osservabilità, per ricostruire tempi ed errori, e un protocollo di valutazione del workflow, descritto separatamente nel Capitolo 8.

1.4 Impostazione del progetto

Il repository umbrella `copernicus_platform` orchestra backend, frontend, server MCP, Redis, Keycloak e harvester NebulaGraph tramite Docker Compose. La domanda viene prima confrontata con il dominio geografico e ambientale, poi trasformata in una rappresentazione strutturata e assegnata alle sorgenti pertinenti. Un grafo LangGraph stabilisce i percorsi consentiti e permette ai rami indipendenti di procedere in parallelo. Rispetto a un agente unico con accesso a tutti gli strumenti, questa struttura rende ispezionabili le condizioni applicate prima di raccogliere i risultati.

Ogni integrazione traduce la risposta remota in un dataset con campi stabili. I client possono così essere provati con input controllati e gli errori restano associati al servizio che li ha prodotti. Il fallimento di un ramo non elimina i risultati già disponibili; la risposta segnala quale fonte non ha risposto.

La memoria è gestita come stato applicativo. A ogni conversazione è associato un checkpoint Redis che comprende i messaggi necessari, i risultati delle sorgenti e i riferimenti agli artefatti. Un turno successivo può così riutilizzare un dataset o aggiornare soltanto la parte indicata dall'utente. La memoria non modifica i parametri del modello e non costituisce una base di conoscenza permanente condivisa fra utenti diversi.

La verifica software copre parsing dei feed, filtri e aggregazioni con test unitari; i percorsi del workflow vengono provati con dipendenze simulate; API, persistenza e operazioni sulla dashboard sono controllate dai test di integrazione. La valutazione end-to-end del comportamento dell'assistente è affidata al protocollo 6.0 (dataset corrente da 105 domande, controlli deterministici e LLM-as-a-Judge), descritto nel Capitolo 8. Dopo le fonti e l'architettura, il Capitolo 6 presenta i percorsi applicativi; i Capitoli 7 e 8 approfondiscono rispettivamente osservabilità e valutazione.

1.5 Perimetro e limiti

Il prototipo non è uno strumento operativo di protezione civile e non sostituisce le comunicazioni delle autorità competenti. GDACS offre valutazioni rapide che possono cambiare quando vengono raccolte nuove informazioni. Le attivazioni CEMS documentano prodotti richiesti e realizzati in uno specifico contesto operativo. Il catalogo STAC indica che determinate osservazioni sono disponibili, non che esse siano sufficienti a stabilire l'entità di un danno.

La dashboard non esegue un'analisi scientifica delle immagini satellitari. La heatmap rappresenta la densità dei centri delle bounding box degli item recuperati, mentre i footprint mostrano le aree dichiarate nei metadati. Non vengono calcolati indici spettrali, classificazioni del suolo o stime automatiche degli effetti di un evento. Tale distinzione limita le conclusioni che possono essere tratte dalle visualizzazioni.

Anche la copertura geografica dipende dai servizi consultati. Un luogo espresso in linguaggio naturale deve essere risolto in coordinate o in una bounding box; nomi ambigui possono richiedere un chiarimento. Ricerca web, cataloghi e feed dipendono inoltre dalla rete e dalla disponibilità dei rispettivi fornitori. Il sistema gestisce errori e risultati parziali, ma non può rendere disponibile una fonte esterna che non risponde.

Per questo le risposte conservano collegamenti e identificativi, distinguono i dati recuperati dal testo generato e segnalano le sorgenti mancanti. Il criterio adottato è rendere riconoscibile il percorso con cui il risultato è stato costruito, anche quando non tutte le fonti sono disponibili.

Capitolo 2

Dall'intelligenza artificiale agli agenti basati su LLM

La progettazione dell'assistente descritto in questa tesi richiede di distinguere il modello linguistico dal sistema che lo utilizza. Un LLM genera testo a partire dal contesto ricevuto; un agente, invece, mantiene uno stato, sceglie strumenti, osserva l'esito delle proprie azioni e decide come proseguire. Il capitolo concentra la discussione sull'apprendimento statistico, sull'architettura Transformer e sui limiti degli LLM isolati, per poi tornare alla nozione di agente adottata nel progetto.

2.1 Intelligenza artificiale e nozione di agente

Russell e Norvig distinguono le definizioni dell'intelligenza artificiale in base al riferimento al pensiero, al comportamento umano o alla razionalità. Per descrivere il sistema sviluppato è sufficiente la loro prospettiva agentiva: un agente riceve percezioni dall'ambiente e sceglie azioni [29]. Questa definizione permette di discutere il comportamento del software senza attribuire al modello proprietà cognitive non osservabili.

Formalmente, un agente può essere rappresentato come una funzione che trasforma la storia delle osservazioni in un'azione. Se $p_1, p_2, \ldots, p_t$ sono le percezioni disponibili fino al tempo t, l'agente calcola una politica:

$$\pi(p_1, p_2, \ldots, p_t) = a_t$$

dove a_t è l'azione scelta. Nell'assistente oggetto della tesi, la storia comprende i messaggi e i risultati restituiti dalle fonti; le azioni sono il routing verso un ramo del workflow, l'invocazione di un tool o la produzione della risposta.

Un classificatore che riceve un'immagine e restituisce una classe rientra nei sistemi di IA, ma non costituisce necessariamente un agente: non interagisce con l'ambiente nel tempo e non conserva un obiettivo persistente. Nell'agente queste due proprietà sono centrali, perché ogni azione modifica le informazioni sulle quali verrà presa la decisione successiva.

2.2 Apprendimento statistico e reti profonde

2.2.1 Dai sistemi simbolici all'apprendimento dai dati

La ricerca degli anni Cinquanta adottò soprattutto rappresentazioni simboliche: fatti, predicati e regole di inferenza venivano specificati esplicitamente. Da questa impostazione derivano, tra l'altro, la pianificazione classica, la dimostrazione automatica di teoremi e i sistemi esperti. Il metodo resta efficace nei domini formalizzabili, ma richiede un aggiornamento manuale quando cambiano regole e conoscenza del dominio.

Il machine learning affronta diversamente il problema: il programmatore definisce una famiglia di modelli e una procedura di apprendimento, mentre i parametri sono stimati a partire dagli esempi. Sistemi simbolici e modelli appresi non corrispondono comunque a fasi mutuamente esclusive; nell'architettura descritta più avanti, le decisioni probabilistiche dell'LLM convivono con regole e schemi di validazione scritti nel codice.

2.2.2 Machine learning

Nel machine learning una funzione sconosciuta viene approssimata a partire dai dati osservati. Dato un insieme di esempi:

$$D = \{(x_i, y_i)\}_{i=1}^{n}$$

si cerca una funzione f_θ parametrizzata da θ tale che $f_\theta(x_i)$ sia vicino a y_i. In classificazione, x_i può essere un'immagine e y_i una classe. In regressione, y_i può essere un valore numerico. In apprendimento non supervisionato, l'obiettivo non è predire etichette,

ma scoprire struttura nei dati: cluster, rappresentazioni compatte, anomalie o fattori latenti.

Rispetto a un programma tradizionale, parte del comportamento non è specificata direttamente nel codice. Il programmatore definisce architettura, procedura di addestramento e funzione obiettivo; il risultato dipende anche dai dati e dai parametri appresi.

Un modello supervisionato viene allenato minimizzando una loss:

$$\theta^* = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} L(f_\theta(x_i), y_i)$$

dove L misura l'errore tra predizione e valore atteso. L'architettura di f_θ, i dati e la funzione di perdita dipendono dal problema, ma i parametri vengono in ogni caso aggiornati in funzione degli esempi osservati.

Il machine learning ha reso possibile trattare problemi in cui scrivere regole esplicite era troppo costoso o fragile. Tuttavia, molti modelli classici richiedevano ancora una fase di feature engineering: un esperto doveva decidere quali caratteristiche estrarre dai dati grezzi. In visione artificiale, per esempio, prima delle reti profonde si progettavano descrittori di bordi, texture o forme. In elaborazione del linguaggio naturale si usavano n-grammi, dizionari, feature sintattiche e rappresentazioni sparse.

2.2.3 Deep learning

Il deep learning estende il machine learning usando reti neurali con molti strati, capaci di apprendere rappresentazioni gerarchiche. Ogni strato trasforma l'input in una rappresentazione intermedia sulla quale opera lo strato successivo. In una rete per immagini, per esempio, i primi livelli possono rilevare pattern locali e quelli finali concetti più vicini alle classi di interesse.

LeCun, Bengio e Hinton descrivono il deep learning come una famiglia di metodi in cui modelli composti da più livelli apprendono rappresentazioni con diversi livelli di astrazione [17]. Le trasformazioni utili al compito vengono apprese dai dati, riducendo il ricorso a feature progettate manualmente.

La disponibilità di dataset estesi e GPU, insieme ai progressi nelle architetture e nelle procedure di ottimizzazione, ha reso praticabile l'addestramento di reti sempre più profonde.

Nel 2012 AlexNet ottenne su ImageNet un errore di classificazione sensibilmente inferiore rispetto agli altri sistemi presentati nella stessa competizione [14]. Il risultato contribuì alla diffusione delle reti profonde nella visione artificiale e mostrò l'effetto congiunto di dataset estesi, GPU e architetture adatte al compito.

Una parte della complessità si sposta così dalle condizioni scritte nel codice ai parametri: il comportamento dipende congiuntamente da dati, architettura e procedura di addestramento.

2.3 Modelli linguistici e architettura Transformer

2.3.1 Modellazione del linguaggio

Il linguaggio naturale presenta ambiguità, dipendenze dal contesto e riferimenti a conoscenze non espresse nella frase. I sistemi di NLP hanno a lungo combinato regole linguistiche, feature manuali e modelli statistici; il deep learning ha poi favorito rappresentazioni dense e reti neurali sequenziali.

Un modello linguistico assegna probabilità a sequenze di token. Se una frase viene rappresentata come $x_1, x_2, \ldots, x_n$, un modello autoregressivo stima:

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, \ldots, x_{t-1})$$

L'addestramento consiste quindi nella predizione del token successivo dato il contesto precedente. Pur essendo formulato come un compito locale, questo obiettivo richiede di rappresentare regolarità grammaticali e semantiche, oltre alle relazioni fattuali presenti nei dati di addestramento.

Il tokenizer converte il testo in unità discrete, che possono coincidere con una parola, una sua parte o un segno di punteggiatura. A ciascuna unità è associato un identificativo numerico, trasformato poi in un embedding sul quale opera il modello.

2.3.2 Transformer e attenzione

L'architettura di riferimento per gli LLM è il Transformer, introdotto da Vaswani et al. in *Attention Is All You Need* [34]. I precedenti modelli sequenziali usavano spesso reti ricorrenti o convoluzionali. Le reti ricorrenti elaborano la sequenza un elemento alla volta e mantengono uno stato nascosto, con minori possibilità di parallelizzare l'addestramento e difficoltà nel rappresentare dipendenze molto lunghe.

Il Transformer usa invece il meccanismo di self-attention. Ogni token costruisce una rappresentazione guardando gli altri token del contesto e pesandoli in base alla loro rilevanza. In forma semplificata, l'attenzione usa tre matrici: query Q, key K e value V. Il punteggio tra un token e un altro viene calcolato confrontando query e key; i value vengono poi combinati con pesi normalizzati:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Il calcolo mette in relazione direttamente posizioni diverse della sequenza, senza comprimere tutto il prefisso in un singolo stato ricorrente. Le diverse teste di attenzione apprendono relazioni differenti in parallelo. Questa struttura facilita la parallelizzazione dell'addestramento e si è dimostrata adatta a scalare con dati e modelli di grandi dimensioni.

2.3.3 Modelli linguistici di grandi dimensioni

Con LLM si indica un modello linguistico neurale addestrato su collezioni testuali estese, in genere basato su Transformer. Il termine "grande" può riferirsi al numero di parametri, ai dati e al calcolo impiegati o alla lunghezza del contesto.

I parametri sono i valori numerici appresi, organizzati principalmente in matrici di pesi e vettori di bias. GPT-3, per esempio, è stato presentato come un modello autoregressivo da 175 miliardi di parametri [3]. La sola dimensione non misura la qualità di un modello: architettura, dati e addestramento incidono sul risultato, mentre l'aumento dei parametri comporta maggiori costi di memoria e calcolo.

Un parametro non corrisponde a una frase memorizzata, ma è un coefficiente che contribuisce a molte predizioni. L'informazione appresa è distribuita tra i pesi, mentre un database conserva record espliciti e li recupera senza generarli. La distinzione è

rilevante per questo lavoro: i dati su eventi e acquisizioni satellitari devono provenire dalle fonti interrogate dal sistema, non dalla memoria parametrica del modello.

Durante l'inferenza, il modello riceve un prompt, cioè una sequenza di input, e produce una distribuzione di probabilità sul token successivo. La generazione procede iterativamente: il token scelto viene aggiunto al contesto e il processo continua. Parametri di decoding come temperatura, top-p o penalità di ripetizione modificano il modo in cui si campiona dalla distribuzione, influenzando determinismo e varietà dell'output.

2.3.4 Pre-addestramento, adattamento e limiti degli LLM isolati

Il ciclo tipico di un LLM moderno comprende il pre-addestramento su grandi collezioni di testo e, spesso, un successivo adattamento alle istruzioni. Nel contesto applicativo di questa tesi, l'uso del solo modello lascia irrisolti tre problemi:

- **Conoscenza statica**: i parametri riflettono il training set e non si aggiornano automaticamente;
- **Allucinazione**: il modello può generare testo plausibile ma falso;
- **Assenza di effetti esterni**: un modello che produce solo testo non può eseguire comandi, leggere documenti aggiornati o interagire con il mondo.

Fonti aggiornate, memoria e azioni devono quindi essere fornite da componenti esterni al modello.

2.4 Agenti basati su LLM

Nelle architetture considerate in questa tesi, un agente IA è un sistema software che usa un LLM per scegliere e coordinare azioni. Attorno al modello è implementato un ciclo di controllo che riceve un obiettivo, osserva il contesto, invoca strumenti esterni e aggiorna lo stato in base ai risultati ottenuti.

La letteratura recente ha studiato esplicitamente l'alternanza tra ragionamento e azione. ReAct combina tracce di ragionamento e invocazioni di strumenti nello stesso processo decisionale [35]; Toolformer affronta invece il problema di apprendere quando e come richiamare API esterne [30]. Il progetto qui presentato non addestra un nuovo

modello: applica questi principi a un workflow LangGraph controllato (*Copernicus Data Workspace*), nel quale le decisioni dell'LLM sono validate da schemi e regole prima dell'esecuzione sulle sorgenti.

Il passaggio dai sistemi simbolici agli agenti contemporanei non va inteso come una sostituzione lineare: regole esplicite e modelli appresi convivono nelle stesse architetture. In questa prospettiva, il modello linguistico è una componente dell'agente e non la sua intera implementazione. Nel prototipo azioni, memoria Redis, artefatti dashboard e controlli restano affidati a componenti software separati. Il capitolo seguente esamina MCP, il protocollo adottato per collegare alcuni di questi componenti a GDACS e CEMS.

Capitolo 3

Model Context Protocol: contesto, strumenti e interoperabilità

3.1 Problema e scelta architetturale

L'assistente sviluppato per questa tesi consulta servizi con interfacce e formati diversi. GDACS pubblica feed RSS in XML, il catalogo CDSE usa richieste STAC e Copernicus EMS restituisce strutture specifiche del servizio. Gestire questi dettagli direttamente nei nodi del grafo avrebbe legato l'orchestrazione e i prompt a trasporto, autenticazione, parsing e messaggi di errore delle singole sorgenti.

Il Model Context Protocol (MCP) è stato adottato per i rami GDACS e Copernicus EMS allo scopo di introdurre un confine tra il workflow e l'accesso ai dati. La specifica definisce un protocollo aperto per collegare applicazioni basate su LLM a strumenti e risorse esterne [19]; la proposta originaria insiste in particolare sulla riusabilità delle integrazioni tra host differenti [1].

Di MCP viene considerato soprattutto il percorso effettivamente usato: host, client, server e invocazione dei tool. I due server realizzati non espongono resources o prompts; queste capacità sono richiamate soltanto per distinguere il perimetro del prototipo da quello della specifica.

MCP propone un modello client-server. L'applicazione IA agisce come host: contiene o coordina il modello, presenta l'interfaccia utente e gestisce le policy. Al suo interno usa uno o più client MCP per connettersi a server MCP. Ogni server espone capacità specifiche: strumenti, risorse o prompt. L'agente può scoprire queste capacità e usarle attraverso messaggi standardizzati.

3.2 Concetti principali

3.2.1 Host

L'host coordina modello, client e policy applicative. Nel caso studiato questo ruolo appartiene al backend FastAPI e al workflow LangGraph; la dashboard nativa fornisce l'interfaccia con l'utente (Open WebUI resta un client legacy opzionale).

È l'host a scegliere quali capacità mostrare al modello. Nel progetto la scelta è ristretta a operazioni di consultazione; nessun server MCP espone scritture o comandi sul sistema.

3.2.2 Client

Il client MCP è il componente che mantiene la connessione con un server. In molte architetture l'utente non lo vede: è una parte interna dell'host. Il client si occupa di inizializzare la sessione, negoziare capacità, inviare richieste e ricevere risposte.

Se l'host usa più server, mantiene più connessioni. Il backend sviluppato in questa tesi utilizza un client comune e apre una sessione distinta per GDACS e Copernicus EMS.

3.2.3 Server

Il server MCP espone funzionalità a cui l'agente può accedere. Può essere locale, per esempio un processo avviato sulla macchina dell'utente, oppure remoto, raggiungibile via rete. Un server può offrire:

- **tools**, cioè azioni invocabili;
- **resources**, cioè dati leggibili e indirizzabili;

- **prompts**, cioè template o procedure testuali riusabili.

La distinzione separa dati, azioni e istruzioni riutilizzabili. Nel progetto la scelta è più circoscritta: i server GDACS e CEMS espongono azioni di lettura e ricerca come tool e non pubblicano resources o prompts.

3.2.4 Trasporto e messaggi

MCP usa messaggi JSON-RPC e mantiene il formato logico separato dal trasporto [19]. La specifica prevede, tra gli altri, processi locali collegati via `stdio` e server raggiunti via HTTP. I servizi GDACS e CEMS del progetto sono container autonomi e comunicano con il backend mediante Streamable HTTP; la modalità `stdio` non è utilizzata.

3.3 Capacità esposte dai server

3.3.1 Tools

Un tool è una funzione descritta da un nome, da una spiegazione del comportamento e dallo schema degli argomenti. Nel server GDACS, ad esempio, la ricerca degli eventi recenti è esposta con un contratto equivalente al seguente:

```
tool: list_recent_events
input: { "hazard": "string", "window": "string" }
output: lista normalizzata di eventi GDACS
```

Descrizione e schema costituiscono le informazioni sulle quali il modello basa la scelta del tool e dei relativi parametri. Ambiguità nel contratto si riflettono quindi direttamente sulla qualità delle invocazioni.

Nel prototipo ogni tool corrisponde a un'operazione della sorgente, accetta un input validato e restituisce errori classificabili. Contratti circoscritti rendono inoltre possibile provare parsing e gestione dei guasti senza coinvolgere il modello linguistico.

3.3.2 Capacità non utilizzate nel prototipo

Una resource rende leggibile un contenuto indirizzabile, di norma identificato da un URI, mentre i prompts sono template testuali per interazioni ricorrenti. La prima descrive

un dato da recuperare; i secondi forniscono all'host una procedura o un'istruzione parametrica. Entrambe le capacità appartengono alla specifica, ma non entrano nel flusso realizzato per GDACS e CEMS.

3.4 Uso di MCP nel workflow

Nel progetto i client scoprono i tool all'apertura della sessione. Il workflow non li presenta tutti indiscriminatamente al modello: il ramo selezionato determina il server e il router specializzato sceglie l'operazione. Dopo la validazione degli argomenti, il client invia la richiesta e il risultato normalizzato rientra nello stato LangGraph. La selezione è quindi distribuita fra decisione del modello e vincoli del grafo.

Quando l'orchestratore seleziona GDACS, il backend non costruisce direttamente l'URL del feed. Il client MCP invia i parametri normalizzati al tool scelto e riceve una risposta strutturata. Il server si occupa della richiesta HTTP, del parsing XML e degli errori della sorgente; il workflow vede soltanto il contratto del tool. Lo stesso schema è usato per le attivazioni Copernicus EMS. MCP non sostituisce quindi le API remote, ma ne isola i dettagli dal resto dell'applicazione.

3.5 Confronto e limiti

3.5.1 Integrazione diretta

MCP non elimina le API custom. Un server MCP, internamente, può chiamare qualunque API: REST, GraphQL, database SQL, SDK proprietari, comandi locali. La differenza è che verso l'applicazione IA espone un'interfaccia uniforme.

Rispetto a un'integrazione collocata interamente nel backend, il costo aggiuntivo è la gestione di sessioni, trasporto e processi separati. In cambio, parsing XML di GDACS e accesso a CEMS restano confinati nei rispettivi servizi. Il client STAC, mantenuto direttamente nel backend, offre un termine di confronto concreto: è più semplice da eseguire, ma rende il package applicativo responsabile dello schema e degli errori dell'API remota.

3.5.2 Tool calling

Il tool calling tradizionale descrive le funzioni che una singola applicazione rende disponibili al modello. MCP opera tra l'host e server separati, dai quali quelle funzioni possono essere scoperte. L'host può quindi presentare al modello un tool ottenuto da un server MCP e tradurre l'invocazione nella corrispondente richiesta di protocollo. I due meccanismi intervengono in punti diversi della stessa catena e non costituiscono soluzioni alternative.

3.5.3 Costi introdotti

MCP non uniforma la semantica delle sorgenti: eventi GDACS e attivazioni CEMS continuano ad avere campi, tempi di aggiornamento e condizioni di errore diversi. Introduce inoltre due servizi da distribuire e monitorare. Per un prototipo con poche funzioni tale costo è visibile; il vantaggio sta nel poter modificare una sorgente senza trasferirne parsing e dipendenze nel workflow.

3.6 Sicurezza e gestione operativa

3.6.1 Autenticazione e autorizzazione

Un protocollo di integrazione non definisce da solo il modello di sicurezza dell'applicazione. Avvio dei server, accesso ai dati, gestione delle credenziali e revoca dei permessi rimangono responsabilità dell'host e dell'infrastruttura.

I server sono raggiungibili sulla rete interna dei container e ricevono configurazione e credenziali dall'ambiente. Il server stabilisce le operazioni disponibili; il workflow limita ulteriormente quelle invocabili nel ramo corrente. Poiché tutti i tool sono di sola lettura, il prototipo non implementa un flusso di conferma per azioni irreversibili.

3.6.2 Prompt injection e dati non fidati

Feed e pagine web possono contenere testo che contraddice le istruzioni dell'applicazione. I contenuti recuperati vengono inseriti come dati e non determinano quali tool siano disponibili: piano, allowlist degli URL GDACS e schemi di output sono controllati

dal codice. La separazione riduce la superficie d'attacco, senza costituire da sola una garanzia contro ogni forma di prompt injection.

3.6.3 Osservabilità

Il client registra servizio, durata, esito e codice diagnostico delle chiamate, senza includere credenziali o prompt completi. MCP rende uniforme il punto nel quale acquisire queste informazioni, ma log e metriche sono implementati dall'applicazione e non dal protocollo.

3.7 Criteri adottati nel progetto

I tool implementati sono piccoli, di sola lettura e dotati di schemi di input espliciti. Le credenziali e gli URL delle sorgenti non vengono inclusi nel contesto del modello; i risultati conservano invece identificativi, timestamp e collegamenti necessari alla verifica. I test dei client coprono risposte vuote, payload non validi, timeout e codici HTTP transitori. Queste proprietà non derivano automaticamente dall'adozione di MCP, ma dalla progettazione dei singoli server e dell'host.

Nel sistema proposto MCP ha un compito delimitato: separa i servizi GDACS e Copernicus EMS dal backend agentico e offre un'interfaccia uniforme per scoprirne e invocarne i tool. Il catalogo satellitare STAC live, usato per acquisizioni e heatmap nel workspace, resta un'integrazione diretta. Anche la Discovery usa un client Nebula e repository applicativi, senza passare da MCP. La presenza di entrambi gli approcci motiva MCP come scelta architetturale concreta, senza attribuirgli proprietà che dipendono dall'applicazione.

Capitolo 4

Fonti informative geospaziali: GDACS e Copernicus

Il progetto integra fonti operative e cataloghi tematici Copernicus. GDACS, CDSE/STAC e CEMS On-demand Mapping descrivono eventi e allerte, acquisizioni satellitari e attivazioni cartografiche di emergenza. Marine e Drought ampliano la scoperta di risorse attraverso un catalogo di metadati a grafo. Confonderle porterebbe a interpretare in modo errato i risultati; una heatmap di acquisizioni, per esempio, non misura l'intensità di un disastro. Il capitolo chiarisce quindi contenuto, modalità di accesso e limiti di ciascuna fonte prima di esaminare l'architettura che le combina.

4.1 Componenti applicativi

L'accesso alle fonti geospaziali è mediato da un workspace conversazionale e da un workflow persistente. I componenti vengono qui raggruppati per funzione, mentre le dipendenze e i nodi del grafo sono descritti nel capitolo successivo.

4.1.1 Interfaccia e API

L'interfaccia primaria è *Copernicus Data Workspace*, servita dal backend su `/dashboard`. È uno spazio di lavoro a schermo intero, con home in italiano e selettore IT/EN, in cui l'utente:

- formula la domanda in linguaggio naturale (Discovery, analisi operativa o percorso combinato);
- avvia una nuova indagine oppure riprende una sessione recente che conserva filtri, artefatti e contesto;
- consulta testo, tabelle e dashboard generate dal workflow.

L'autenticazione è OIDC tramite Keycloak (realm `pecilli`). Il client pubblico usato dal browser è `dashboard-client`. I ruoli distinguono almeno un profilo amministratore, con accesso anche agli strumenti di valutazione, e un profilo utente standard.

Open WebUI resta disponibile soltanto come client legacy opzionale (profilo Compose `legacy-ui`): una Pipe può inoltrare la chat al backend nel formato delle completion OpenAI, ma non è richiesta dal percorso standard [22]. Il viewer cartografico STAC (`/copernicus`) resta disponibile come vista dedicata, distinta dalla dashboard analitica.

FastAPI espone l'endpoint di chat, le API degli artefatti (lettura, mutazioni, revisioni, chat contestuale) e lo streaming SSE. I modelli Pydantic validano richieste, decisioni degli agenti e risultati normalizzati prima che passino al nodo successivo [27]. Il formato HTTP resta così separato dalla rappresentazione interna dello stato.

4.1.2 Orchestrazione e memoria

LangChain gestisce messaggi, invocazioni e output strutturati del modello [15]; LangGraph dispone le operazioni in un grafo con rami condizionali e punti di convergenza [16]. La scelta linguistica determina il piano, ma è il grafo a stabilire quali nodi possano essere eseguiti e quando i risultati siano pronti per la risposta.

Redis conserva il checkpoint associato alla conversazione: messaggi utili, risultati raccolti, filtri e riferimenti agli artefatti. Un follow-up come "raggruppali per giorno" può quindi operare sul dataset precedente senza interrogare di nuovo le sorgenti [28]. Nel percorso ordinario la chiave deriva dal `thread_id` della sessione workspace (e, nel client legacy, dagli identificativi utente/chat di Open WebUI). Il fallback basato sul primo messaggio, previsto per client privi di metadati, non offre un isolamento adeguato a un'installazione multiutente.

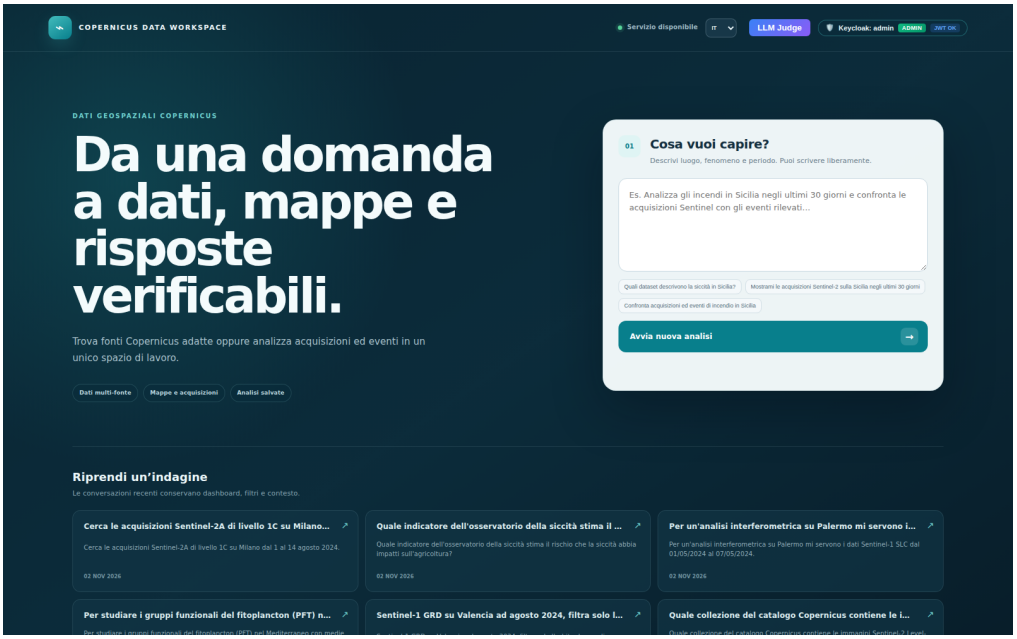


Figura 4.1: Home della dashboard nativa (*Copernicus Data Workspace*): ingresso della domanda, suggerimenti, ripresa delle indagini e stato del servizio autenticato con Keycloak.

4.1.3 Modello e distribuzione

Il modello servito da Ollama è remoto rispetto al backend (nel prototipo documentato, tipicamente `qwen3:8b` per il workflow e un modello distinto per il Judge). Riceve il contesto preparato dai nodi e interviene nell'interpretazione, nel piano e nella risposta; non accede direttamente a GDACS, Copernicus o Redis e non calcola i valori dei grafici. Le chiamate sono soggette a timeout, retry selettivi e limiti di concorrenza [21].

Docker Compose, nella root del repository umbrella, coordina backend, asset frontend, Keycloak, Redis, servizi MCP e infrastruttura NebulaGraph sulla rete `pecillispace`. I profili `legacy-ui`, `harvest` e `tools`, insieme all'overlay di osservabilità, aggiungono le rispettive funzioni [9, 25]. La Tabella 4.1 riporta la responsabilità assegnata a ciascun elemento dello stack.

Tecnologia	Ruolo nel sistema
Copernicus Data Workspace	Home, chat, artefatti Discovery/operativi e ripresa sessioni.
Keycloak	Autenticazione OIDC, realm `pecilli` e ruoli applicativi.
LangChain	Comunicazione strutturata tra agenti e modello linguistico.
LangGraph	Coordinamento dei passaggi, delle fonti e dei follow-up.
Redis Stack	Checkpoint per conversazione e riuso dei risultati.
NebulaGraph	Catalogo condiviso di metadati Marine, STAC e Drought.
Ollama	Inferenza del modello usato dal workflow (e, se configurato, dal Judge).
FastAPI e Pydantic	API del sistema e controllo della forma dei dati scambiati.
Docker Compose	Avvio coordinato dei servizi sulla rete `pecillispace`.
Prometheus e Grafana	Raccolta delle metriche e diagnosi aggregata LLMOps.
OpenTelemetry e Phoenix	Tracciamento delle singole esecuzioni.
Open WebUI (opzionale)	Client legacy di chat; profilo Compose `legacy-ui`.

Tabella 4.1: Tecnologie del nucleo applicativo, del catalogo e dell'osservabilità.

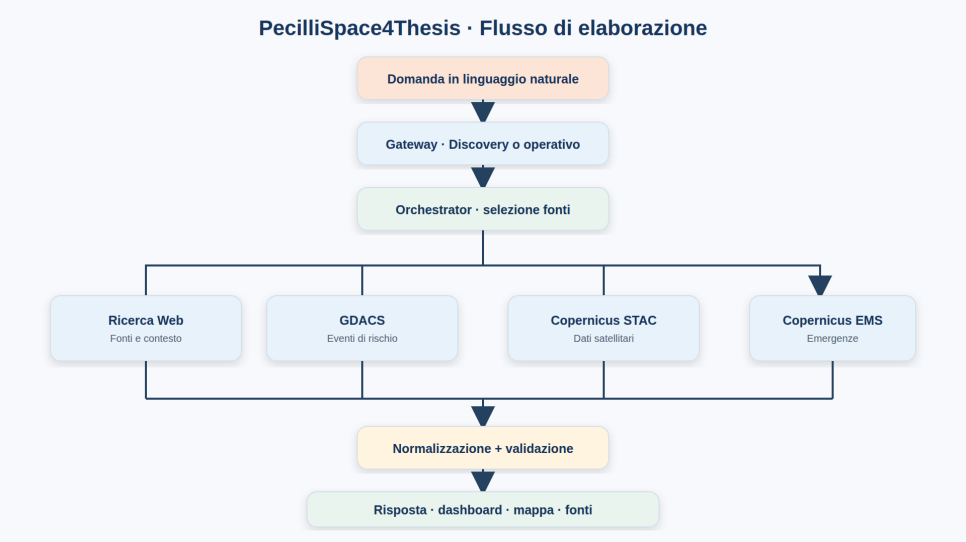


Figura 4.2: Flusso di elaborazione sintetico: dalla domanda in linguaggio naturale alla risposta con dashboard, mappa e fonti. Il gateway distingue Discovery e percorso operativo prima della selezione delle sorgenti.

4.2 GDACS

4.2.1 Scopo e contesto operativo

GDACS, acronimo di *Global Disaster Alert and Coordination System*, è un quadro di cooperazione tra le Nazioni Unite, la Commissione europea e gli attori internazionali della gestione delle emergenze. Il suo obiettivo principale è ridurre le carenze informative e di coordinamento nella prima fase successiva a grandi disastri improvvisi [12].

La piattaforma raccoglie dati sugli eventi, esegue o riceve stime automatiche dell'impatto potenziale e distribuisce notifiche. La sua funzione di *early awareness* mette in relazione caratteristiche fisiche dell'evento, popolazione esposta, vulnerabilità e capacità di risposta. I modelli variano in base al tipo di pericolo e producono una valutazione sintetica utile a stabilire quali eventi richiedano attenzione internazionale [11].

4.2.2 Eventi e livelli di allerta

GDACS pubblica informazioni su diverse categorie di pericolo, tra cui terremoti, possibili tsunami, cicloni tropicali, alluvioni, eruzioni vulcaniche, siccità e incendi. I record disponibili contengono normalmente:

- identificativo e tipologia dell'evento;
- titolo, paese e coordinate;
- istante di pubblicazione o aggiornamento;
- livello di allerta;
- misure specifiche, come magnitudo o severità;
- collegamento alla scheda ufficiale.

I livelli verde, arancione e rosso esprimono una stima crescente dell'impatto e della rilevanza internazionale. Non devono essere interpretati come una semplice misura fisica del fenomeno. Per un terremoto, ad esempio, il livello considera anche esposizione, vulnerabilità e capacità di risposta; una magnitudo elevata in un'area poco popolata non equivale automaticamente a un impatto umanitario elevato.

4.2.3 Accesso programmatico

I dati e le stime GDACS sono accessibili tramite feed RSS e altri formati standard, così da poter essere integrati in portali e applicazioni esterne [12]. Sono disponibili feed distinti per intervallo temporale e tipologia di evento; la documentazione indica, ad esempio, feed relativi alle ultime 24 ore, all'ultima settimana e a specifiche soglie per i terremoti [10].

Nel progetto il feed viene interrogato da un servizio MCP autonomo. Il servizio normalizza il contenuto XML e restituisce oggetti strutturati. Il backend non espone direttamente il feed al modello linguistico: l'agente riceve invece un contratto stabile e filtra i record per luogo, periodo, tipo di evento e, quando richiesto, soglie quantitative.

4.2.4 Limiti

Una notifica GDACS è una stima rapida e può cambiare quando arrivano nuovi dati. Non sostituisce le autorità locali, una valutazione sul campo o un bollettino ufficiale di protezione civile. Alcuni processi sono automatici, altri includono dati e valutazioni provenienti da istituzioni o fonti esterne. Per questo il progetto conserva link, timestamp e identificativi: la risposta dell'agente deve restare verificabile e non presentare una stima preliminare come un dato definitivo.

4.3 Il programma Copernicus

Copernicus è il programma di osservazione della Terra dell'Unione europea. Mette a disposizione missioni satellitari, dati, infrastrutture di accesso e servizi tematici. Per questa tesi sono rilevanti percorsi complementari:

1. il *Copernicus Data Space Ecosystem* (CDSE), usato per cercare metadati relativi alle acquisizioni satellitari;
2. il *Copernicus Emergency Management Service* (CEMS), usato per consultare attivazioni e prodotti cartografici di emergenza;
3. Copernicus Marine, per individuare prodotti, dataset e variabili oceanografiche;

4. i servizi Drought consultati dall'harvester, per scoprire layer, coverage e disponibilità temporali relativi alla siccità.

Il catalogo satellitare descrive osservazioni disponibili; CEMS documenta invece attivazioni operative e prodotti derivati dall'analisi di immagini e di altre informazioni.

4.4 Copernicus Data Space Ecosystem e STAC

4.4.1 Ricerca delle acquisizioni

Il CDSE offre API per catalogazione, download, visualizzazione e processamento dei dati di osservazione della Terra. Fra le API di catalogo è disponibile STAC, insieme a OData e Sentinel Hub Catalog API [4].

STAC, *SpatioTemporal Asset Catalog*, definisce un modello interoperabile per descrivere collezioni e singoli elementi geospaziali. Un elemento associa un identificativo a geometria, intervallo temporale, proprietà, collegamenti e asset [32]. La ricerca può quindi combinare:

- una o più collezioni, ad esempio Sentinel-2 Level-2A;
- un intervallo temporale;
- un'area espressa come bounding box;
- filtri su attributi, quali copertura nuvolosa o piattaforma;
- un limite al numero di risultati.

Il catalogo CDSE implementa STAC 1.1.0 ed espone un endpoint di ricerca standardizzato [5].

4.4.2 Dal metadato alla heatmap

Il ramo STAC del progetto non esegue una classificazione scientifica delle immagini e non produce una mappa dei danni. Recupera invece i metadati delle acquisizioni che intersecano luogo e periodo richiesti. STAC distingue `bbox` e `geometry`: da ogni Item si usa la bounding box del prodotto (rettangolo), se ne calcola il centro e si costruisce:

- una heatmap della densità dei centri delle bounding box dei risultati recuperati (non della copertura areale né dell'intensità del fenomeno);
- i footprint rettangolari della bounding box, selezionabili;
- un pannello con identificativo, collezione, data, piattaforma, copertura nuvolosa, asset e collegamenti disponibili.

I KPI di conteggio si riferiscono agli *item* STAC restituiti dalla ricerca (metadati di prodotto), non a tile, scene ottiche distinte o osservazioni scientifiche derivate. La visualizzazione non misura la severità del disastro né la frazione di territorio effettivamente coperta dalle geometrie precise degli Item.

4.5 Copernicus Emergency Management Service

CEMS fornisce informazioni geospaziali a supporto della risposta alle emergenze e della gestione del rischio. È uno dei sei servizi Copernicus e comprende monitoraggio e allerta precoce, cartografia on demand e informazioni sull'esposizione [8].

Nel progetto viene usata la componente *On-demand Mapping*. Le attivazioni sono identificate da codici come `EMSR884` o `EMSN204`. Il prefisso consente di distinguere due percorsi principali:

Rapid Mapping (`EMSR`). Produce in tempi rapidi informazioni geospaziali per la fase immediatamente successiva a un evento. Una attivazione può comprendere più aree di interesse e più prodotti [6].

Risk and Recovery Mapping (`EMSN`). Supporta prevenzione, preparazione e ricostruzione mediante analisi geospaziali più articolate, basate su immagini ottiche e radar [7].

Il relativo server MCP espone operazioni per elencare le attivazioni, ottenere una sintesi e recuperare i dettagli di una specifica attivazione. I risultati normalizzati includono codice, nome, categoria, paesi, fase di gestione del rischio, stato, data, aree di interesse e prodotti disponibili.

4.6 Marine e Drought nel catalogo di Discovery

La richiesta di una risorsa scientifica richiede metadati più articolati del solo elenco delle acquisizioni. Per esempio, studiare la temperatura superficiale del Mar Nero implica individuare un dataset, la variabile disponibile, l'area coperta e la risoluzione temporale. La Discovery usa un catalogo locale alimentato da tre harvester, documentati nella guida tecnica del progetto [25].

4.6.1 Metadati Marine

Il collector Marine legge il catalogo con `copernicusmarine.describe(show_all_versions=True)`. Conserva la gerarchia tra prodotto, dataset, versione, parte e servizio di accesso, oltre a variabili e coordinate. Una variabile può essere descritta mediante nome, *standard name*, unità e descrizione; gli assi mediante estremi, passo e numero di valori. Gli array completi delle coordinate vengono esclusi.

Un secondo collector ispeziona le funzioni pubbliche dell'SDK per documentare operazioni e parametri. Il catalogo può quindi associare una risorsa alle modalità con cui sarebbe selezionabile. Registrare l'operazione `subset` descrive una possibilità di accesso; l'harvester non la invoca per acquisire i file scientifici.

4.6.2 Metadati Drought

Il collector Drought consulta `GetCapabilities` WMS e `DescribeCoverage` WCS. Raccoglie servizi, layer e sotto-layer, coverage ed estensioni temporali. Queste descrizioni consentono di verificare se una risorsa sia compatibile con tema e periodo richiesti senza scaricare ogni mappa o raster. Il percorso non usa `GetCoverage`; riguarda le coverage previste dal catalogo applicativo e non costituisce un censimento universale di tutte le risorse sulla siccità.

4.6.3 Due usi distinti di STAC

L'harvester CDSE salva root del catalogo, collezioni, campi interrogabili e, quando previsto, item e riferimenti agli asset. La finestra recente predefinita per gli item è di

due giorni, con limiti configurabili. La persistenza di un collegamento a un asset non comporta il download del file.

Nella Discovery l'agente legge questi metadati da NebulaGraph; nel percorso operativo il client interroga CDSE STAC per cercare acquisizioni. I risultati operativi non devono quindi essere interpretati come una semplice vista degli item già presenti nel grafo. Aggiornamento del catalogo locale e disponibilità del servizio remoto sono proprietà separate.

4.7 Confronto e complementarità

Aspetto	GDACS	CDSE/STAC	Copernicus CEMS
Domanda	Quali eventi richiedono attenzione?	Quali acquisizioni satellitari coprono area e periodo?	Quali attivazioni e prodotti di emergenza sono disponibili?
Unità	Evento e allerta	Item e collezione satellitare	Attivazione, area di interesse e prodotto
Uso	Monitoraggio di eventi	Heatmap e footprint delle acquisizioni	Prodotti Rapid Mapping e Risk/Recovery
Integrazione	Server MCP GDACS	Client STAC nel backend	Server MCP CEMS

Tabella 4.2: Ruoli delle fonti del percorso operativo. Marine e Drought alimentano inoltre il catalogo di Discovery descritto nella Sezione 4.6.

Le tre sorgenti rispondono a domande diverse e possono essere combinate. Per una richiesta su un'alluvione, GDACS fornisce gli eventi e il livello di allerta; CEMS può indicare un'attivazione di cartografia; STAC può mostrare quali acquisizioni satellitari sono presenti nell'area e nel periodo. Una ricerca web aggiunge infine contesto recente e fonti narrative.

4.8 Implicazioni per la progettazione dell'agente

Gli agenti specializzati convertono XML, risposte STAC e record CEMS in dataset con campi stabili. In questo modo i client possono essere provati senza coinvolgere il modello,

i rami indipendenti possono procedere in parallelo e il fallimento di una sorgente non cancella i risultati già disponibili.

La normalizzazione non rende equivalenti le fonti. Il workflow conserva semantica e provenienza di ogni record anche quando GDACS, CDSE/STAC e CEMS sono interrogati nello stesso turno. Su questa separazione si basa l'architettura del capitolo successivo.

Anche la rappresentazione comune nel grafo preserva le differenze tra dataset, variabile, collezione e layer. Una risorsa scoperta indica quali dati potrebbero sostenere uno studio; non è già il risultato scientifico di quello studio.

Capitolo 5

Architettura del sistema proposto

5.1 Obiettivo e requisiti

Il sistema proposto è un assistente conversazionale specializzato in geografia, osservazione della Terra ed emergenze naturali, esposto come *Copernicus Data Workspace*. Deve interpretare richieste in linguaggio naturale, interrogare più fonti, produrre testo e visualizzazioni e mantenere il contesto nei turni successivi, anche riprendendo sessioni già avviate.

I requisiti che guidano l'architettura sono:

- autenticare l'accesso al workspace (OIDC) e associare memoria e artefatti a un `thread_id` di sessione;
- applicare un confine di dominio prima di effettuare chiamate esterne;
- selezionare una o più fonti in funzione della richiesta;
- distinguere Discovery sul grafo Nebula, STAC live sul catalogo remoto e, quando serve, il percorso combinato;
- eseguire in parallelo i rami operativi indipendenti;
- mantenere separati ragionamento, acquisizione dati e presentazione;
- conservare memoria e artefatti per follow-up e ripresa sessioni;

- presentare le acquisizioni Copernicus in una dashboard interattiva e modificabile senza alterare i dati di origine;
- continuare a fornire un risultato quando una sola sorgente fallisce;
- esporre un'API compatibile con OpenAI Chat Completions (SSE), usata dal workspace e riutilizzabile da client legacy;
- rendere osservabili errori, tempi e dipendenze;
- consentire la valutazione del workflow con riferimenti verificabili, controlli oggettivi e rubriche esplicite.

5.2 Vista architetturale

L'applicazione segue una struttura a componenti. La dashboard nativa gestisce l'interazione con l'utente (chat, ripresa sessioni, artefatti Discovery e acquisizioni) dopo l'accesso OIDC via Keycloak; FastAPI espone gli endpoint della chat e degli artefatti; LangGraph coordina gli agenti; i servizi MCP e i client dedicati accedono alle fonti esterne. La dashboard analitica delle acquisizioni non è un testo generato dal modello: è un artefatto HTML che il browser carica tramite API e rende a partire da un contratto JSON validato.

Il catalogo NebulaGraph aggiunge una base di metadati condivisa fra le conversazioni, distinta dai checkpoint Redis. Gli harvester aggiornano il catalogo; il backend lo consulta attraverso repository controllati. Metriche e trace osservano l'esecuzione, mentre il runner del benchmark invoca lo stesso workflow per valutarlo [25].

Il modello linguistico non comunica direttamente con i servizi remoti. Produce decisioni strutturate che il codice applicativo valida prima di invocare le integrazioni e acquisirne il risultato.

I ruoli dello stack sono già riepilogati nella Tabella 4.1. In questa architettura LangGraph e Redis costituiscono il nucleo stateful, FastAPI espone il servizio e Pydantic delimita i contratti; MCP, STAC, ECharts e Leaflet intervengono invece nei punti di integrazione e presentazione indicati in Figura 5.2.

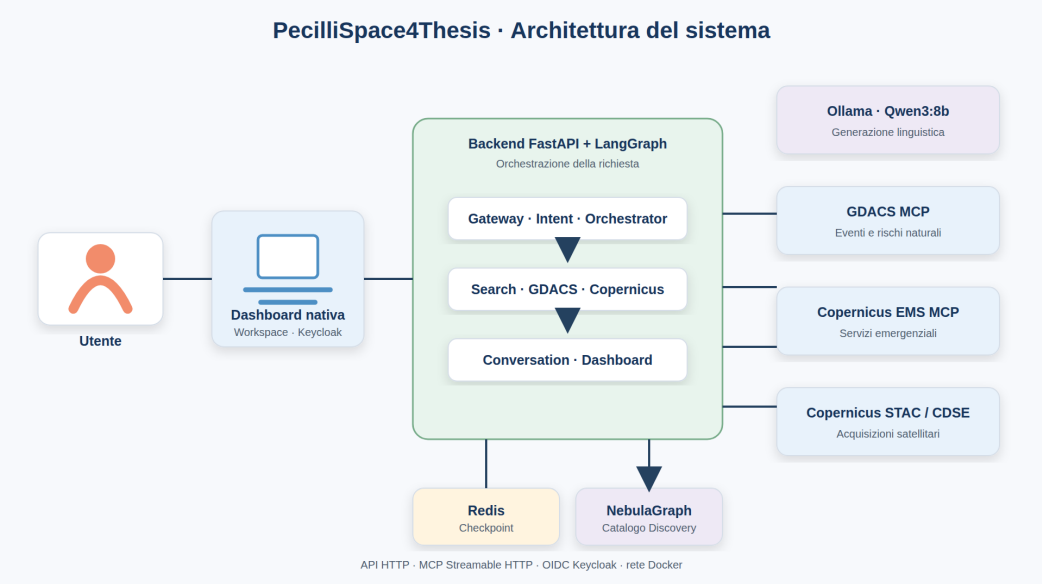


Figura 5.1: Vista a componenti del prototipo: dashboard nativa autenticata, backend LangGraph, Redis, NebulaGraph e sorgenti MCP/STAC sulla rete Docker condivisa.

5.3 Organizzazione del backend e del frontend

Il backend è suddiviso secondo responsabilità esplicite:

`agents/` gateway, parsing dell'intento, orchestrazione, ricerca, GDACS, Copernicus STAC/CEMS, Discovery, grafici, dashboard e conversazione;

`api/` route FastAPI per chat, salute, artefatti (lettura, mutazioni, revisioni, chat contestuale) e valutazioni;

`core/` stato condiviso, classe base, registro, errori e hook di osservabilità;

`integrations/` client CDSE/STAC, trasformazione geospaziale e client di lettura Nebula;

`repositories/` query del catalogo a grafo;

`models/` modelli Pydantic e contratti di scambio;

`services/` MCP, risoluzione di luogo e tempo, Discovery, artefatti, mutazioni, health check;

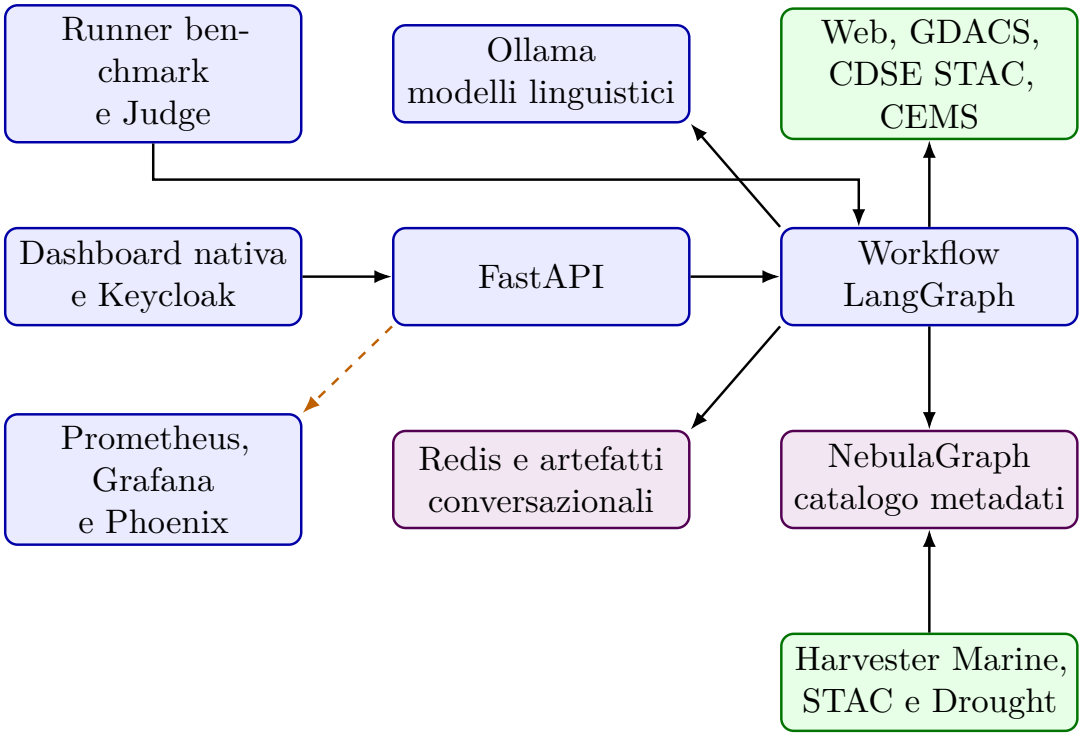


Figura 5.2: Componenti applicativi, catalogo e strumenti di analisi. La Discovery legge NebulaGraph; il ramo operativo consulta le fonti remote. Il benchmark invoca il workflow, mentre la telemetria ne osserva l'esecuzione.

`evals/` dataset, runner, Judge e statistiche;

`workflow.py` costruzione del grafo LangGraph.

Il frontend applicativo vive in `frontend/dashboard/` e viene servito dal backend. I moduli principali separano chat, workspace, filtri, grafici, mappa, tabelle, revisioni, confronti e pannello valutazioni. Il browser usa ECharts per i grafici e Leaflet per la mappa; l'interfaccia è bilingue (IT/EN). Il viewer STAC dedicato resta in `frontend/copernicus_viewer/`.

La strumentazione è raccolta nel pacchetto `copernicus_llmops` (`observability/llmops/`); gli harvester e lo schema Nebula sono sotto `copernicus-harvester/`. I due server MCP sono esterni al package applicativo e possiedono dipendenze e container autonomi: un problema del feed GDACS non richiede di modificare il codice del ramo CEMS o STAC.

5.4 Il grafo degli agenti

5.4.1 Percorso principale

Il `WorkflowAgent` è il punto di ingresso logico. Una nuova richiesta attraversa una parte comune e viene instradata verso Discovery o verso il percorso operativo. Nel secondo caso il piano seleziona i worker necessari.

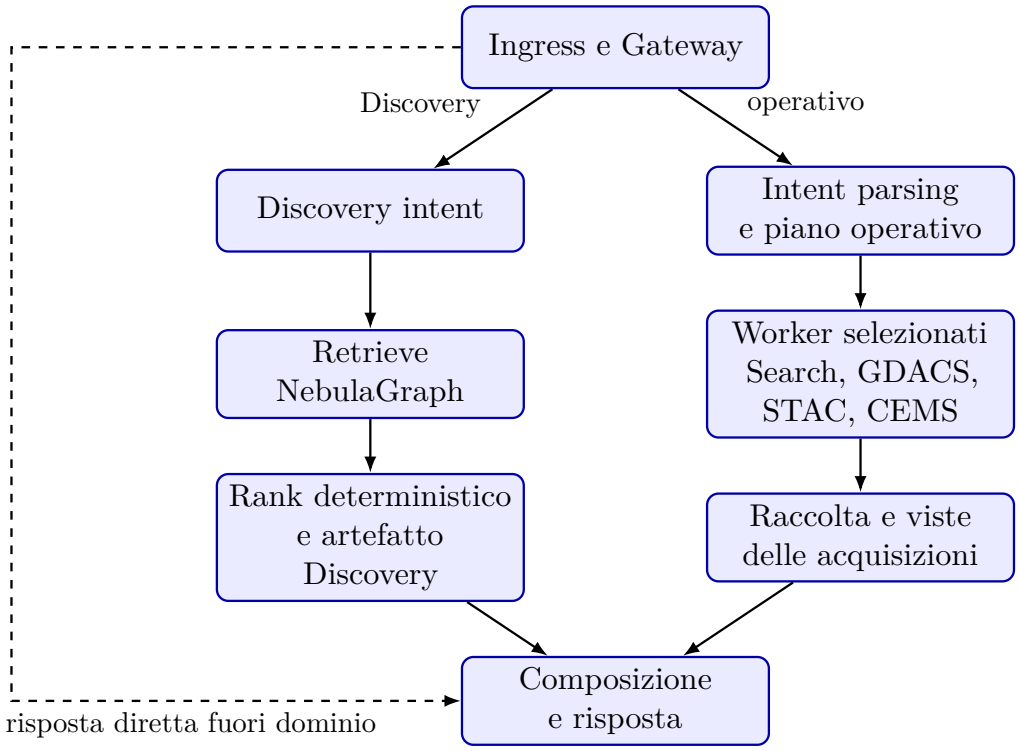


Figura 5.3: Due percorsi di una nuova richiesta ammessa. Il disegno sintetizza i rami e omette i dettagli dei follow-up e dei nodi finali di verifica.

Nel ramo operativo il router attiva solo un sottoinsieme delle fonti. LangGraph usa oggetti `Send` per creare un `source_worker` per ogni sorgente selezionata. Search, GDACS e STAC possono quindi procedere contemporaneamente. Quando sono richiesti insieme CEMS e STAC, il sistema esegue prima CEMS: l'area o le coordinate dell'attivazione possono rendere più precisa la successiva ricerca satellitare.

5.4.2 Gateway e confine di dominio

Il Gateway Agent classifica ogni turno, non soltanto il primo. Sono ammessi argomenti relativi a geografia, spazio, osservazione della Terra, clima, pericoli naturali ed emergenze. Una domanda estranea riceve una risposta diretta e non raggiunge parser, motore di ricerca o dashboard.

Se la classificazione non è disponibile, il sistema adotta un comportamento *fail closed*: non avvia integrazioni esterne. La scelta riduce chiamate non necessarie e impedisce che un errore del modello allarghi implicitamente il dominio operativo.

5.4.3 Parsing dell'intento

L'Intent Parsing Agent converte il testo in una `ResolvedRequestSchema`. Lo schema contiene:

- argomenti e relativa categoria;
- luoghi testuali, paese, città, coordinate o bounding box;
- intervallo temporale normalizzato e fuso orario;
- obiettivo dell'analisi, filtri e aggregazioni;
- modalità di risposta e viste richieste.

Espressioni quali "oggi", "la settimana scorsa" o "negli ultimi 30 giorni" vengono trasformate in estremi temporali. Se manca un'informazione indispensabile, il parser produce una richiesta di chiarimento; la domanda parziale viene conservata e completata al turno successivo.

5.4.4 Orchestratore e piano di esecuzione

L'Orchestrator Agent restituisce una lista di sorgenti, non una singola classe. Il piano ammette `search`, `gdacs`, `copernicus` e `copernicus_ems`. Alla decisione del modello si affiancano regole deterministiche. Nel percorso operativo multi-sorgente qui descritto, Search e Copernicus STAC forniscono contesto web e acquisizioni per la dashboard;

questa regola non implica l'attivazione di tali sorgenti nella Discovery. Le richieste di pericolo aggiungono GDACS, mentre un codice CEMS esplicito aggiunge il ramo dedicato alle attivazioni. Il workflow può inoltre applicare GDACS come fonte strutturata di supporto quando il piano segnala un possibile pericolo.

Il risultato è una forma di orchestrazione ibrida: l'LLM gestisce la varietà del linguaggio, mentre regole verificabili preservano i vincoli applicativi.

5.4.5 Il ramo Discovery

La Discovery usa i nodi `discovery_intent`, `discovery_retrieve` e `discovery_rank`. L'agente d'intento rappresenta tema, area, periodo, tipo di risorsa e variabili. Il servizio `services/copernicus_discovery.py` espande alias di dominio, seleziona le sorgenti e interroga `CopernicusGraphRepository`. Controlli di compatibilità filtrano i candidati sulla base dei metadati disponibili. Un'area incompatibile non giustifica la sostituzione silenziosa del luogo; eventuali alternative temporali richiedono un avviso.

Il ranking è deterministico: il metodo `CopernicusDiscoveryRankingAgent.rank()` applica la politica `graph-score-v1`, senza sottoporre i candidati a un LLM. Lo score è calcolato in `CopernicusDiscoveryService._score()` su feature verificabili nei metadati (corrispondenza di titolo/identità/variabili, compatibilità spaziale e temporale, tipo di risorsa, servizio preferito), con penalità per incompatibilità e normalizzazione finale in $[0, 1]$. La selezione ammette candidati con punteggio almeno $\max(0{,}28, \text{best} - 0{,}20)$ e al più 30 risorse; se sono richieste più variabili esplicite, viene forzata la presenza di un rappresentante per ciascuna. Metadati assenti riducono i bonus disponibili senza inventare copertura. Il nome "Agent" identifica una responsabilità del workflow, non un componente che effettua inferenza linguistica.

Il repository limita ogni interrogazione a 120 righe; il servizio conserva al massimo 30 candidati prima del ranking. Il ranking può quindi ordinare soltanto ciò che il recupero gli consegna: qualità del retrieval e qualità dell'ordinamento restano distinte. La ground truth del benchmark usa spesso un target unico; una domanda di Discovery può però ammettere più risorse valide, e la mancata citazione del target è critica soprattutto quando l'unicità è giustificata dai metadati. La risposta e l'artefatto riportano identificativi, metadati e collegamenti delle risorse selezionate [25].

5.5 Catalogo a grafo e pipeline di harvesting

5.5.1 Modello del dominio

NebulaGraph conserva vertici, archi orientati e proprietà. I tag descrivono i tipi dei vertici; gli edge type descrivono le relazioni. Lo space applicativo predefinito nel Compose è `pecillispace`. La struttura logica riunisce i tre cataloghi sotto `service:copernicus`, collegato mediante `HAS_SOURCE` a Marine, CDSE/STAC e Drought.

Sottografo	Entità e relazioni principali
Marine	Product, Dataset, Version, Part, DataService, Variable e Coordinate; Operation e Parameter documentano l'accesso.
CDSE/STAC	StacCatalog, StacCollection, StacQueryable, StacItem e StacAsset; gli asset conservano riferimenti ai file.
Drought	OgcService, DroughtLayer, DroughtCoverage e DroughtTimeExtent; sono rappresentate gerarchie dei layer e disponibilità temporali.

Tabella 5.1: Articolazione del catalogo condiviso di metadati.

Gli identificativi deterministici hanno prefissi di dominio per separare le sorgenti. Un dataset Marine è collegato alle sue variabili direttamente e attraverso il contesto di versioni e servizi di accesso. La radice comune permette la navigazione fra cataloghi, ma non costituisce un'ontologia di equivalenze fra ogni variabile Marine e ogni indicatore Drought.

5.5.2 Raccolta, aggiornamento e riconciliazione

Ogni harvester raccoglie e normalizza i metadati in un `GraphPayload`, poi il repository esegue upsert di vertici e archi. Le definizioni operative dello schema sono nei file `marine/app/graph/schema.ngql`, `space/app/schema.ngql` e `drought/app/schema.ngql`, relativi a `copernicus-harvester/`. I file numerati sotto `nebula/schema/` dello stesso harvester sono segnaposto nella revisione documentata: non vanno confusi con gli schemi effettivamente applicati.

I campi `first_seen_at`, `last_seen_at` e `last_harvested_at` descrivono prima osservazione, ultima presenza e ultima elaborazione; `metadata_hash` permette di riconosce-

re cambiamenti. La riconciliazione può mantenere risorse scomparse con `active=false`. Per una vista corrente occorre filtrare sia vertici sia archi attivi.

Sono presenti protezioni contro rimozioni anomale: la soglia predefinita è il 25%, applicata a categorie Marine con almeno 20 vertici, collezioni STAC con almeno 10 e layer Drought con almeno 5. Gli item STAC sono accumulati per finestre incrementali: l'assenza in una finestra recente non ne prova la scomparsa dal catalogo globale. Un limite di sicurezza raggiunto deve emergere come errore, anziché essere presentato come raccolta completa.

L'aggiornamento a batch non offre un'atomicità globale dell'harvest: un guasto può lasciare alcuni batch già salvati. Inoltre gli upsert aggiornano proprietà correnti, senza conservare tutte le revisioni precedenti. Timestamp e flag di attività rendono lo stato ispezionabile, ma non permettono di ricostruire qualsiasi snapshot storico.

La pianificazione del profilo `harvest` prevede Marine alle 02:00, STAC alle 03:00 e Drought alle 03:30, con fuso predefinito Europe/Rome. Lock, timeout, tentativi limitati e file di stato controllano l'esecuzione; la salute del catalogo richiede anche la freschezza dell'ultimo job riuscito, oltre alla raggiungibilità del database [25].

5.5.3 Infrastruttura e confine di lettura

NebulaGraph separa gestione dello schema (`metad`), conservazione dei dati (`storaged`) ed esecuzione delle query (`graphd`) [20]. Il Compose usa la versione predefinita 3.8.0, un servizio per ruolo e volumi persistenti per metadati e storage. Lo space creato dal client Marine prevede dieci partizioni, identificativi `FIXED_STRING(256)` e `replica_factor=1`: la separazione dei processi non fornisce da sola alta disponibilità.

I container si collegano a `nebula-graphd:9669`; dall'host la porta è pubblicata secondo `NEBULA_HOST_PORT`. Il servizio `nebula-storage-activator` registra lo storage e termina con successo; il suo stato normale è quindi un processo concluso. Occorre inoltre allineare `NEBULA_SPACE`: il default Marine fuori Compose è `copernicus`, diverso da quello del backend.

La Discovery usa query predisposte dal codice, tramite `repositories/copernicus_graph.py` e `integrations/nebula/client.py`. `NebulaReadClient` ammette i prefissi di lettura `MATCH`, `GO`, `FETCH` e `SHOW`, blocca istruzioni multiple e mutazioni fuori

dai letterali e valida i parametri. Il modello non consegna al database uno script nGQL arbitrario. Il vincolo è applicativo: l'utente predefinito del Compose è `root`, quindi non dimostra una restrizione equivalente dei privilegi del database.

5.6 Agenti specializzati e nodi di elaborazione

Nel codice molti componenti portano il suffisso "Agent", ma solo alcuni usano un LLM per decidere. La Tabella 5.2 distingue nodi decisionali linguistici da servizi e trasformazioni deterministiche, così che il titolo multiagentico non venga letto come "ogni nodo ragiona con un modello".

Componente	LLM?	Decide cosa fare?	Operazione principale
Gateway	Sì	Sì (dominio/route)	Classificazione della richiesta
Discovery intent	Sì	Sì (intento catalogo)	Estrazione tema/area/periodo
Intent / Orchestrator	Sì	Sì (piano fonti)	Parametri e selezione sorgenti
Discovery retrieve	No	No	Query Nebula parametrizzate
Discovery rank	No	Secondo regole fisse	Ordinamento `graph-score-v1`
Worker fonti	Ibrido	Esecuzione del piano	Recupero Search/GDACS/STAC/CEMS
Chart / Validator	No	No	Aggregazioni e riparazione viste
Conversation	Sì	Sì (sintesi/follow-up)	Composizione della risposta

Tabella 5.2: Componenti del workflow: uso di LLM, potere decisionale e operazione. "Ibrido" indica un nodo che può usare il modello per parametri, ma esegue recupero e filtri nel codice.

In sintesi, gli agenti decisionali linguistici sono concentrati in gateway, parsing/intento, orchestrazione e conversazione; ranking, repository, filtri STAC e validazione delle viste sono elaborazioni deterministiche.

5.7 Contratti e stato condiviso

Gli agenti decisionali producono structured output validato da Pydantic. Questa scelta separa due linguaggi:

- JSON tipizzato per lo scambio interno;
- Markdown in linguaggio naturale per l'utente.

Lo stato LangGraph estende `MessagesState` e contiene richiesta risolta, piano, azione di follow-up, risultati delle quattro fonti, grafici, dashboard e risposta finale. Per la dashboard conserva inoltre filtri collegati, stato visuale, revisioni e conversazioni contestuali per settore. Ogni campo è associato a un `thread_id`; conversazioni diverse non condividono quindi la memoria.

Il ramo Discovery aggiunge intento, risorse recuperate e selezionate e riferimento al proprio artefatto. Questi dati appartengono al contesto della conversazione; il catalogo NebulaGraph resta invece una base persistente condivisa, aggiornata dagli harvester.

I risultati delle sorgenti seguono un'impostazione comune: stato dell'operazione, dataset normalizzato, aggregazioni, fonti ed eventuali informazioni di errore. Il Conversation Agent non deve conoscere il formato originale di RSS, STAC o CEMS.

5.8 Persistenza

La persistenza separa due bisogni: riprendere una conversazione e conservare artefatti troppo grandi per il prompt. Nessuno dei due meccanismi sostituisce da solo i controlli di autorizzazione dell'utente autenticato.

5.8.1 Checkpoint conversazionali

Redis Stack è l'unico backend della memoria conversazionale. Il `AsyncRedisSaver` di LangGraph salva messaggi e stato dopo i nodi del grafo. A ogni nuova richiesta il checkpoint del relativo `thread_id` viene ricaricato automaticamente. Se Redis non è configurato o non è raggiungibile, il backend non viene dichiarato pronto.

La cronologia attiva è limitata agli ultimi messaggi utili, mentre i risultati strutturati restano nello stato, evitando una crescita indefinita del prompt.

5.8.2 Archivio degli artefatti

Le heatmap e i payload della dashboard possono essere più grandi della memoria testuale. Sono quindi salvati come artefatti JSON in un archivio dedicato. Lo stato conserva l'identificativo; il browser legge l'artefatto con `GET /api/dashboards/{id}` e invia mutazioni controllate a endpoint dedicati. Ogni modifica crea una revisione che può essere confrontata, annullata o ripristinata. In caso di indisponibilità temporanea di CDSE, il servizio può cercare un artefatto precedente compatibile per area, collezione e intervallo temporale, dichiarandone il riuso.

5.9 Dashboard generativa e verificabile

Nel percorso operativo il workflow produce una dashboard analitica a partire dai soli record STAC Copernicus normalizzati; nel workspace essa compare come artefatto del percorso "Analisi dati Copernicus". La Discovery presenta invece risorse del catalogo a grafo ("Ricerca dati Copernicus"). Search, GDACS e CEMS rimangono fonti della risposta testuale o di sezioni dedicate e non alimentano le serie quantitative della dashboard STAC. La provenienza dei valori resta così esplicita e non vengono confrontate nello stesso grafico grandezze eterogenee.

L'artefatto contiene KPI, grafici a barre, linee o torta, tabella, heatmap e provenienza: campi usati, filtri attivi, metodo di aggregazione, numero di record e identificativi dei prodotti. L'LLM può scegliere o pianificare una vista, ma il backend ricalcola conteggi, percentuali, medie, variazioni e confronti sui record; non accetta valori numerici prodotti liberamente dal modello.

I controlli consentono di cambiare tipo di grafico, ordinamento, Top N, visibilità e filtro. Un click su grafico o tabella può attivare un cross-filter: backend e interfaccia ricalcolano coerentemente grafici, KPI, tabella e heatmap. La chat contestuale di ciascun settore risponde sul risultato corrente e può richiedere una mutazione tipizzata; usa la ricerca web solo quando la domanda richiede contesto esterno non deducibile dai dati visualizzati.

5.9.1 Leggibilità e correttezza dei grafici

La dashboard applica regole di rendering per evitare che una visualizzazione formalmente corretta diventi illeggibile. I grafici temporali, quelli con più di otto categorie e il grafico primario occupano l'intera riga; gli altri sono disposti in una griglia responsiva. Per molte categorie sono disponibili scorrimento e zoom; le etichette vengono troncate, ruotate quando necessario e rese non sovrapposte. Le torte mostrano etichette esterne soltanto fino a sei categorie e usano una legenda scorrevole; le barre orizzontali aumentano dinamicamente l'altezza. Il renderer osserva inoltre i cambiamenti di dimensione del contenitore e ridimensiona ECharts dopo il caricamento e quando la sezione viene riaperta.

Queste regole non sostituiscono la validazione semantica: prima di salvare una modifica, il Validator riallinea etichette, valori e serie e converte valori non validi in valori numerici sicuri. Ogni grafico offre anche una tabella dei dati e l'esportazione CSV, così che il risultato visivo resti verificabile.

5.10 Integrazione MCP

I server GDACS e CEMS usano FastMCP e il trasporto Streamable HTTP. Il backend scopre e invoca i tool tramite un client comune. Gli strumenti principali sono:

- elenco degli eventi GDACS recenti, filtrato per pericolo e finestra;
- dettaglio di un evento GDACS ufficiale;
- catalogo delle operazioni CEMS;
- elenco, sintesi e dettaglio delle attivazioni CEMS.

Gli URL e le credenziali restano nella configurazione dei servizi e non vengono passati al modello. Il client applica timeout; i servizi applicano retry limitati agli errori transitori, evitando di ripetere richieste errate.

5.11 Distribuzione tramite container

La distribuzione comprende servizi applicativi e catalogo condiviso:

1. dashboard nativa (asset statici serviti dal backend);
2. Keycloak per OIDC e realm `pecilli`;
3. backend FastAPI;
4. server MCP GDACS;
5. server MCP CEMS;
6. Redis Stack;
7. NebulaGraph con metad, storaged, graphd e attivatore dello storage.

Il modello Ollama è raggiunto come dipendenza esterna. I server MCP sono visibili sulla rete interna; l'utente accede alla dashboard su `http://localhost:8080/dashboard` e, se attivato, al client legacy Open WebUI. Il backend dipende anche dal completamento dell'attivatore; le dipendenze usano health check e condizioni di avvio. Il profilo `harvest` abilita job e schedulatori, mentre l'overlay `observability/docker-compose.observability.yml` aggiunge Prometheus, Grafana e Phoenix e avvia il backend con strumentazione. Liveness e readiness sono separate; un processo vivo ma privo di memoria non viene considerato pronto.

5.12 Sicurezza e osservabilità

Il sistema applica misure coerenti con un agente che usa fonti esterne e un workspace multiutente:

- accesso al workspace protetto da Keycloak (OIDC) con ruoli applicativi;
- credenziali e URL remoti forniti tramite variabili d'ambiente;
- validazione degli URL di dettaglio GDACS, limitati a host ufficiali;

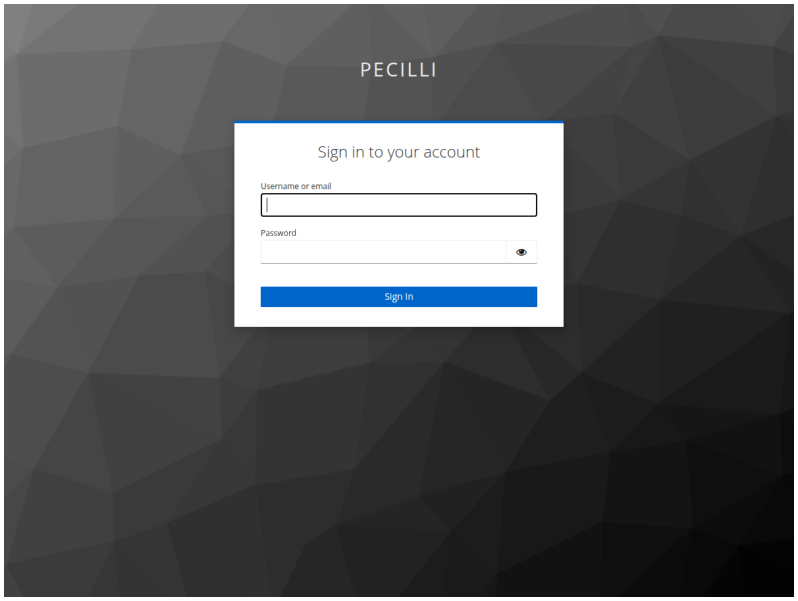


Figura 5.4: Schermata di accesso Keycloak al realm `pecilli`, intermedia fra il browser e la dashboard nativa.

- schemi Pydantic per rifiutare forme non valide (non la correttezza semantica di luogo o periodo);
- nessuna azione distruttiva esposta dai tool MCP;
- contenuti esterni trattati come dati, non come istruzioni;
- filtraggio dei dati sensibili nei log applicativi;
- correlazione fra risposta e log tramite `request_id`/`thread_id` e header diagnostici.

Un `thread_id` distinto isola i checkpoint Redis per conversazione, ma non dimostra da solo l'autorizzazione dell'utente autenticato su ogni artefatto: tale garanzia richiede controlli di accesso espliciti oltre alla sola chiave di sessione. Analogamente, la "riparazione" di valori non numerici nelle viste serve a evitare grafici malformati; non autorizza a sostituire dati mancanti con zeri semanticamente falsi.

I log strutturati includono `request_id`, `thread_id`, servizio, durata, codice di errore e possibilità di retry. Gli endpoint `/health/live`, `/health/ready` e `/health?deep=true` distinguono stato del processo, dipendenze obbligatorie e controlli remoti.

L'overlay osserva inoltre HTTP e SSE, workflow, nodi, chiamate LLM, tool MCP e repository del grafo. Prometheus conserva le metriche e Grafana le visualizza; OpenTelemetry esporta i trace a Phoenix. I trace possono catturare prompt e output secondo configurazione e limiti: la protezione dei log non implica l'assenza di contenuti nella telemetria. Il Capitolo 7 descrive copertura, interpretazione e limiti di questo livello.

Il vincolo trasversale dell'architettura riguarda i dati quantitativi: l'LLM può selezionare un percorso o proporre una vista, ma non costituisce mai la sorgente dei valori mostrati. Acquisizione, filtri, calcoli e validazione restano passaggi eseguiti dal codice sui record recuperati.

Capitolo 6

Funzionamento end-to-end, casi d'uso e verifica

Per collegare l'architettura al comportamento osservabile, il capitolo segue alcune richieste dall'ingresso in *Copernicus Data Workspace* fino alla risposta. Gli scenari sono ricavati dai contratti e dai rami implementati nel codice; i valori citati illustrano il flusso e non costituiscono risultati relativi a un evento reale. I risultati di benchmark documentati sono presentati separatamente nel Capitolo 8.

6.1 Ingresso della richiesta

Dalla home del workspace l'utente descrive luogo, fenomeno e periodo e avvia una nuova analisi; può anche riprendere un'indagine precedente che conserva dashboard, filtri e contesto. La dashboard invia i messaggi al backend mediante l'endpoint compatibile con le chat completion OpenAI. La richiesta contiene modello, cronologia, opzione di streaming e metadati utili a ricavare l'identificativo della conversazione. Il backend anonimizza e normalizza tale identificativo, quindi invoca il `WorkflowAgent`.

Nel client legacy Open WebUI possono comparire anche richieste tecniche generate dall'interfaccia (titolo della chat, suggerimenti). Questi task non devono essere scambiati per domande geografiche e vengono gestiti separatamente.

Il nodo `ingress` estrae l'ultimo messaggio umano e ripulisce eventuali marcatori

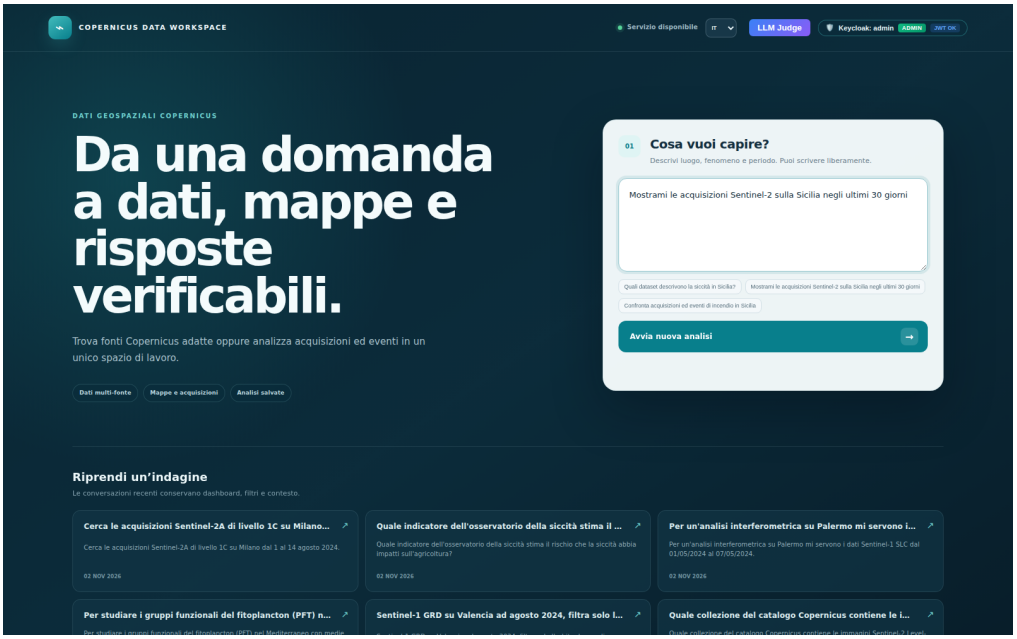


Figura 6.1: Formulazione di una richiesta operativa nella dashboard nativa, con suggerimenti contestuali e avvio di una nuova analisi.

dell'interfaccia. Il Gateway decide quindi se procedere. Da questo punto sono possibili tre casi:

1. nuova richiesta ammessa: parsing e routing completi;
2. follow-up ammesso: riuso del checkpoint e selezione dei soli nodi necessari;
3. richiesta fuori dominio o classificazione fallita: risposta diretta.

6.2 Scenario 1: analisi multi-sorgente di un pericolo

Si consideri la domanda:

> *"Mostrami i terremoti di magnitudo superiore a 3 avvenuti in Italia negli ultimi sette giorni e fammi vedere la copertura satellitare."*

6.2.1 Risoluzione semantica

Il parser produce una richiesta strutturata simile alla seguente:

- categoria: `hazard`, sottotipo terremoto;
- luogo: Italia, con bounding box quando il geocoding è disponibile;
- periodo: intervallo assoluto corrispondente agli ultimi sette giorni;
- filtro: magnitudo maggiore di 3;
- viste: dati strutturati e mappa.

Il valore temporale non viene lasciato come testo ambiguo. È risolto rispetto all'istante corrente e al fuso orario del backend. Anche il luogo viene trasformato, quando possibile, da etichetta a geometria.

6.2.2 Pianificazione ed esecuzione

Poiché la categoria è un pericolo, il piano comprende Search, GDACS e Copernicus STAC. I tre worker possono iniziare in parallelo:

- Search recupera contesto web attuale;
- GDACS chiede al server MCP gli eventi e applica i filtri;
- STAC interroga CDSE per area, periodo e collezioni.

Il filtro GDACS combina tutte le condizioni. Se sono presenti coordinate, la verifica spaziale usa la bounding box del luogo invece di affidarsi soltanto al nome del paese. Una bounding box nazionale (per esempio dell'Italia) è un rettangolo che può includere aree esterne al territorio: è un filtro geometrico, non una verifica di appartenenza amministrativa. Per una richiesta cittadina il sistema non degrada automaticamente l'ambito all'intero Stato. Periodo, magnitudo, tipo e posizione devono essere soddisfatti dallo stesso record. La risposta si limita agli eventi presenti nel feed interrogato: non è un censimento completo di tutti i fenomeni nella regione.

Il client STAC invia una ricerca con bounding box e intervallo ISO 8601. Per intervalli o aree estesi può suddividere la richiesta in partizioni temporali e quadranti, con limiti sul numero di tentativi. I risultati sono deduplicati per identificativo e ridotti ai metadati utili.

6.2.3 Sincronizzazione e risposta

LangGraph attende la convergenza dei worker. Il Chart Agent riceve soltanto dataset normalizzati e aggregazioni; il renderer della mappa riceve footprint e centroidi. Nel percorso operativo, le acquisizioni STAC alimentano inoltre un artefatto dashboard incorporato nel workspace. Il Conversation Agent compone infine:

1. sintesi testuale;
2. eventuali tabelle;
3. dashboard Copernicus con KPI, grafici, tabella e heatmap;
4. elenco delle fonti consultate.

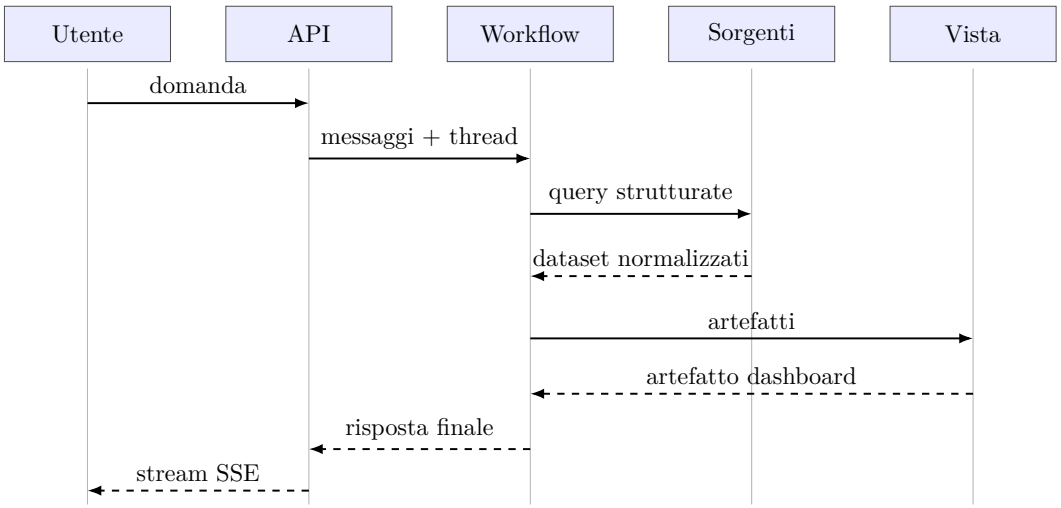


Figura 6.2: Sequenza end-to-end di una richiesta multi-sorgente. L'artefatto dashboard è costruito nel workflow a partire dai record STAC normalizzati; la vista nel browser lo carica e lo presenta, senza ricalcolarne i valori.

6.3 Scenario 2: interrogazione di una attivazione CEMS

La domanda:

> *"Dammi i dettagli dell'attivazione Copernicus EMSR884."*

contiene un identificativo non ambiguo. Il ramo CEMS risponde alla domanda diretta; nel prototipo attuale il piano operativo può comunque attivare anche Search e STAC come *arricchimento* per contesto e dashboard. È una scelta di prodotto, non una necessità semantica della sola richiesta di dettaglio: a parità di domanda, aumenta latenza e dipendenze. Il CEMS Router valida il codice con il formato `EMSR###` o `EMSN###` e sceglie l'endpoint corretto.

Per `EMSR` vengono consultati i dettagli Rapid Mapping; per `EMSN` quelli Risk and Recovery. Il dataset restituito può includere paese, categoria, stato, istante di attivazione, aree di interesse, numero e tipo dei prodotti, URL di download o mappe.

Se una successiva domanda chiede anche le acquisizioni Sentinel nella medesima area, il ramo CEMS può essere eseguito prima di STAC. Le coordinate o l'estensione dell'attivazione vengono usate per arricchire la ricerca, evitando un geocoding generico.

6.4 Scenario 3: follow-up e riuso della memoria

Dopo una prima risposta l'utente può scrivere:

> *"Ora raggruppali per giorno e mostrami un grafico a barre."*

Il messaggio dipende dal contesto: il pronome "li" si riferisce al dataset precedente. Il Gateway riceve una vista compatta dello stato e riconosce la continuità del dominio. Il Conversation Agent pianifica quindi una delle azioni di follow-up:

- `answer_from_context`, per spiegare dati già presenti;
- `render_chart` o `render_heatmap`, per una nuova vista dello stesso dataset;
- aggiornamento di una singola sorgente;
- aggiornamento di più sorgenti;
- `new_request`, se l'utente cambia oggetto.

Nel caso dell'esempio non è necessario ripetere le chiamate GDACS e STAC. Il Chart Agent usa il dataset conservato in Redis e aggiorna la visualizzazione. Se l'utente opera

nella dashboard, il Composer traduce il comando in una mutazione tipizzata e il backend salva una nuova revisione dell'artefatto. Questo comportamento riduce latenza, traffico verso servizi esterni e possibilità che un aggiornamento involontario renda incoerenti due turni consecutivi.

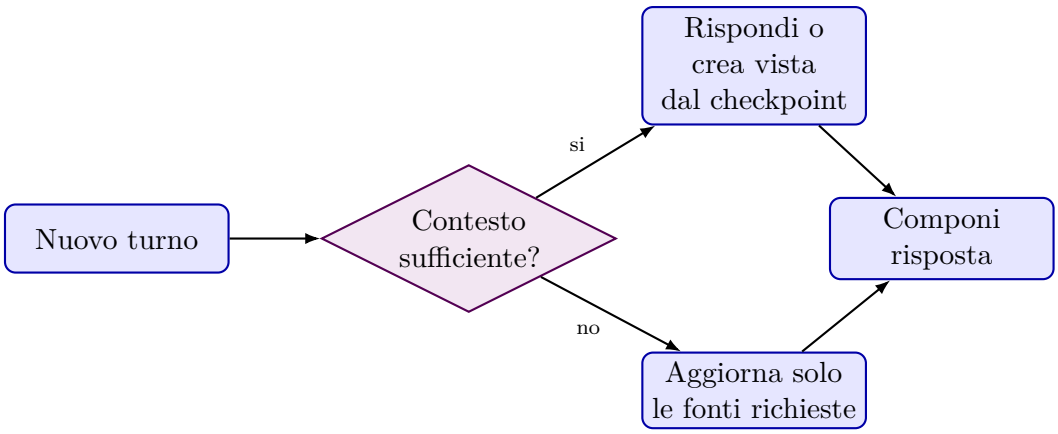


Figura 6.3: Scelta fra riuso della memoria e aggiornamento delle fonti.

6.5 Scenario 4: scoprire un dataset nel grafo

Si consideri la richiesta:

> *"Cerco un dataset orario di temperatura superficiale del Mar Nero: quale variabile contiene?"*

Il Gateway deve riconoscere un compito di Discovery. Il nodo d'intento rappresenta il tema temperatura superficiale, l'area Mar Nero, la risoluzione oraria e il tipo dataset con variabile. Il servizio recupera candidati dal sottografo Marine, ne verifica la compatibilità e applica il ranking deterministico descritto nella Sezione 5.4.5.

Lo snapshot di riferimento del benchmark, salvato il 13 ottobre 2026 e riportato nella guida [25], associa questa ricerca al dataset `cmems_obs-sst_blk_phy-sst_nrt_diurnal-oi-0.0625deg_PT1H-m`, dal titolo *Black Sea Diurnal Subskin SST Analysis, L4, 1/16deg hourly*, con variabile `analysed_sst`. La bounding box registrata è [26,375; 38,75; 42,375; 48,8125] nell'ordine ovest, sud, est, nord. Sono metadati del-

lo snapshot; non costituiscono una nuova lettura del database né una misura della temperatura.

La risposta deve rendere riconoscibili dataset, variabile e provenienza. L'artefatto consente di consultare le risorse selezionate; non va popolato con acquisizioni Sentinel come se fossero osservazioni della variabile Marine. L'esempio distingue così la scoperta della fonte scientifica dalla ricerca di scene satellitari e dall'eventuale analisi dei valori, che resta fuori perimetro.

Se un turno successivo chiede le acquisizioni relative alla stessa area, il passaggio al percorso operativo deve conservare il riferimento geografico. Verificare questa continuità richiede di confrontare intento, query effettiva, artefatto persistito e viste rese dal browser. Il solo cambio della scheda visuale non prova che sia stata eseguita una ricerca corretta. Questo controllo conversazionale è distinto dai casi a thread nuovo del benchmark.

6.6 Dashboard Copernicus

La dashboard analitica è il canale principale per le visualizzazioni delle acquisizioni satellitari. Nel ramo operativo viene prodotta e aperta nel workspace come artefatto a sé (vista "Analisi dati Copernicus"), con navigazione dedicata a risultato, grafici, mappa, dati e attività, più una chat laterale sul risultato già caricato. Grafici, KPI, tabella e heatmap usano esclusivamente record STAC Copernicus normalizzati; Search, GDACS e CEMS restano nella risposta testuale o in sezioni separate. La scelta evita di mescolare, per esempio, il numero di eventi GDACS con il numero di prodotti satellitari.

Il ramo Discovery presenta invece un artefatto del catalogo: schede e metadati delle risorse pertinenti. Le due viste rispondono a compiti diversi e devono mantenere esplicita la provenienza.

6.6.1 Dati, provenienza e composizione

Il Chart Agent costruisce una specifica JSON con valori derivati dal dataset: etichette, serie, tipo di grafico, campo e misura di origine, filtri, numero di record e identificativi dei prodotti. Il modello può proporre il tipo di vista o una nuova aggregazione, ma

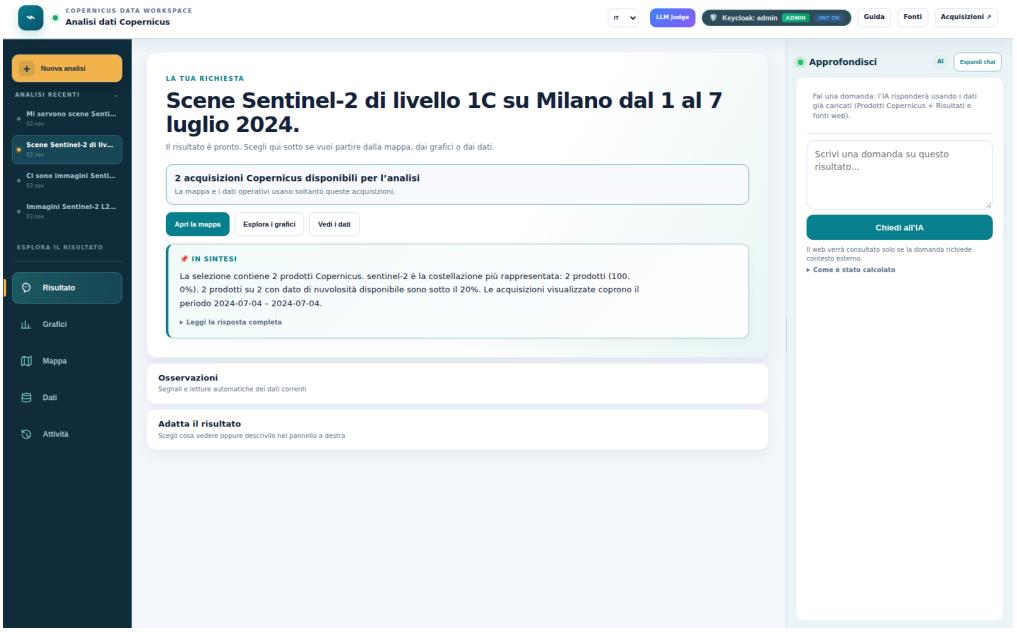


Figura 6.4: Sessione operativa ripresa dal workspace: sintesi delle acquisizioni (conteggio di prodotti/item STAC recuperati), navigazione Risultato/Grafici/Mappa/Dati e chat laterale sul risultato già caricato.

conteggi, percentuali, medie, variazioni e confronti sono sempre ricalcolati dal backend. La sezione "Come è stato calcolato" e la tabella dei dati di ciascun grafico rendono questa provenienza ispezionabile; i dati possono inoltre essere esportati in CSV.

La composizione iniziale può limitare gli output richiesti dall'utente: una domanda come "solo la heatmap" non apre inutilmente grafici e tabella, mentre una richiesta generale include KPI, grafici, tabella, sintesi e mappa. Anche in assenza di acquisizioni l'artefatto viene mostrato con uno stato vuoto esplicito e non con un grafico artificialmente popolato.

6.6.2 Grafici leggibili e senza sovrapposizioni

Le regole di rendering descritte nella Sezione 5.9.1 vengono applicate alla specifica generata prima che la dashboard sia mostrata. Il Dashboard Validator riallinea etichette, valori e serie e intercetta valori non numerici; ECharts gestisce poi dimensioni, etichette e scorrimento nel browser. La verifica del payload rimane così separata dalle correzioni puramente grafiche.

6.6.3 Interazione, revisioni e chat contestuale

Nel workspace i controlli visivi permettono di scegliere barre, linee o torta, ordinamento, Top N e visibilità dei grafici. Un click su categoria o cella può attivare un cross-filter: il backend applica il filtro ai record e ricalcola simultaneamente KPI, grafici, tabella e heatmap. I filtri attivi sono mostrati e possono essere azzerati con un solo comando.

Ogni modifica passa attraverso azioni tipizzate, validate e persistite. Sono disponibili annullamento, ripristino e confronto tra revisioni. La chat contestuale sul risultato ha un thread indipendente per grafico, tabella, heatmap o dashboard: risponde sul sottoinsieme corrente e può trasformare istruzioni come "filtra solo Sentinel-2A" o "cambia questo grafico in linee" in mutazioni. La ricerca web è usata solo quando serve un contesto esterno non deducibile dai dati visualizzati. La home del workspace conserva inoltre l'elenco delle indagini recenti, così da riprendere una sessione senza ripetere la domanda iniziale.

6.6.4 Heatmap Copernicus

Leaflet rappresenta una heatmap della densità dei centri delle bounding box degli item recuperati e collega il risultato a una vista con footprint rettangolari e metadati STAC. Non misura copertura areale né intensità del fenomeno osservato. Se un cross-filter è attivo, la mappa usa lo stesso sottoinsieme di record degli altri componenti.

6.7 Streaming della risposta

Durante l'esecuzione il backend invia aggiornamenti sullo stato dei nodi (interpretazione, routing, recupero delle fonti, composizione). FastAPI traduce gli eventi LangGraph in Server-Sent Events nel formato delle chat completion OpenAI; il workspace li mostra come avanzamento della richiesta, senza esporre il ragionamento interno del modello.

Il testo generato dal Conversation Agent viene post-processato prima della consegna definitiva. Tabelle Search/GDACS, fonti e dashboard Copernicus sono aggiunti in un ordine stabile. Lo stream pubblica la versione finale, non una bozza priva dell'artefatto; al termine viene inviato il marcatore di chiusura.

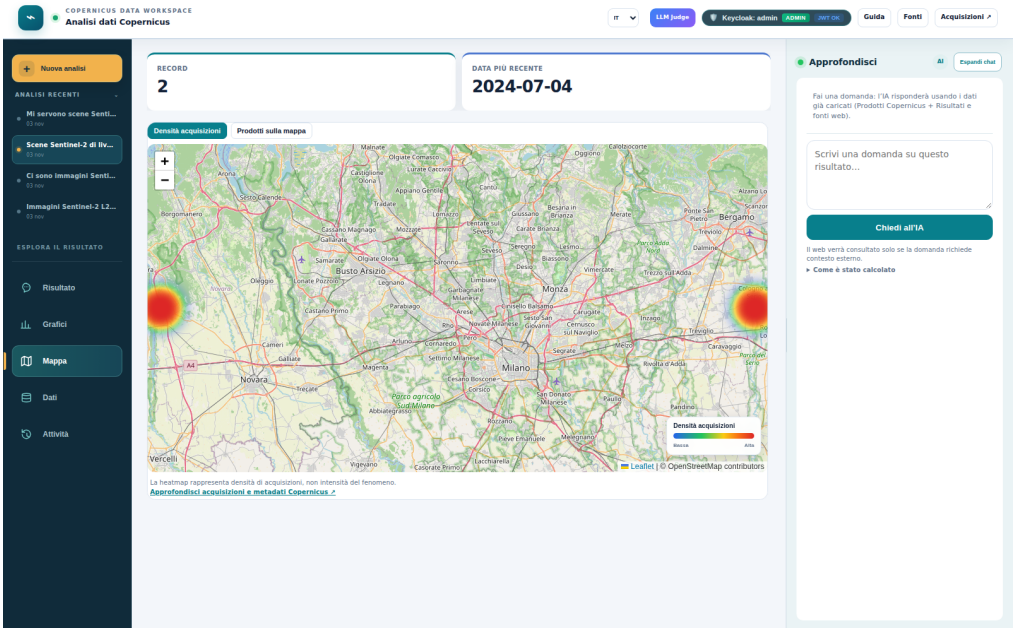


Figura 6.5: Vista *Mappa*: densità dei centri delle bounding box degli item STAC recuperati (non intensità del fenomeno) e chat laterale sul risultato.

6.8 Gestione degli errori

6.8.1 Errore di una sorgente

Ogni chiamata a una sorgente è racchiusa da un contratto di errore. Se GDACS non risponde ma Search e STAC terminano correttamente, il grafo conserva i risultati disponibili. Il Conversation Agent può produrre una risposta parziale e segnalare con precisione la dipendenza mancante.

Anche all'interno di STAC una ricerca partizionata può fallire soltanto per alcune aree o finestre temporali. I prodotti ottenuti restano utilizzabili, ma la limitazione di copertura deve essere propagata nella risposta e nell'artefatto. Una tabella non vuota non dimostra che tutte le partizioni abbiano risposto né che il catalogo sia stato esplorato completamente.

Gli errori sono classificati in configurazione, autenticazione, rete, timeout, risposta non valida e persistenza. A ciascuno sono associati:

- codice pubblico stabile;

- identificativo diagnostico univoco;
- servizio coinvolto;
- indicazione sulla possibilità di ripetere l'operazione;
- dettaglio tecnico destinato soltanto ai log.

6.8.2 Timeout e retry

I retry sono applicati soltanto agli errori presumibilmente transitori, come timeout, problemi di rete, HTTP 429 e HTTP 5xx. Gli errori HTTP 4xx non vengono ripetuti automaticamente. Anche il modello remoto dispone di timeout, backoff e limite di concorrenza, così che i rami paralleli non saturino il runner Ollama.

6.8.3 Errore della memoria

Redis è una dipendenza obbligatoria. L'assenza del checkpointer potrebbe far apparire valido un follow-up privo del contesto precedente; il progetto preferisce quindi non avviarsi in modalità degradata. L'endpoint readiness restituisce errore finché memoria e grafo non sono pronti.

6.9 Matrice dei percorsi e delle evidenze

La Tabella 6.1 collega percorsi tipici alle fonti previste. La Tabella 6.2 precisa, per le funzionalità descritte nel capitolo, quale tipo di evidenza è disponibile: molte capacità del prodotto (GDACS, CEMS, follow-up, autenticazione, ripresa sessione) non sono misurate dal benchmark del Capitolo 8, limitato a casi `graph` e `acquisitions`.

6.10 Strategia di verifica

La verifica è organizzata per livelli.

Richiesta	Rami principali	Risultato atteso
Clima in Sicilia	Piano operativo	Contesto web e acquisizioni secondo il piano selezionato.
Dataset orario SST del Mar Nero	Discovery su NebulaGraph	Risorsa Marine, variabile e metadati di compatibilità.
Siccità in Sicilia e acquisizioni	Discovery + operativo	Risorse Drought/Marine e copertura STAC nella stessa sessione.
Terremoti in Indonesia	Search + GDACS + STAC	Eventi filtrati, contesto web e copertura delle acquisizioni.
Dettagli EMSR884	CEMS (+ Search/STAC opz.)	Attivazione e prodotti; arricchimento dashboard se abilitato.
Grafico dei risultati	Chart su contesto	Nuova vista senza recuperare nuovamente i dati.
Aggiornamento GDACS	Refresh GDACS	Sostituzione del solo dataset eventi.
Ripresa sessione dal workspace	Checkpoint Redis	Ricarico di filtri, artefatti e cronologia utili.
Domanda fuori dominio	Gateway	Risposta diretta, senza chiamate esterne.

Tabella 6.1: Esempi di instradamento del workflow e di ripresa sessione.

Funzionalità	Test mock	Integrazione	Benchmark	Browser
Discovery (grafo)	Parziale	Harvest/repo	Sì (51)	Artefatto
STAC live	Query/filtri	Client CDSE	Sì (54)	Dashboard
GDACS / CEMS / web	Normalizz.	MCP/HTTP	No	Scenario
Follow-up / revisioni	Workflow	API mutazioni	No	Scenario
Keycloak / sessioni	—	OIDC	No	Accesso
Osservabilità	Metriche	Overlay	No	Grafana

Tabella 6.2: Funzionalità descritte e tipo di evidenza disponibile. "Benchmark" si riferisce al protocollo 6.0 del Capitolo 8; "Browser" indica verifica manuale o screenshot, non misurata dal runner.

6.10.1 Test unitari

I test unitari controllano normalizzazione dei feed GDACS, sicurezza degli URL, query STAC, generazione di grafici, contratti di errore e trasformazioni temporali. Per la dashboard verificano inoltre la riparazione di valori e serie non validi, il ricalcolo del cross-filter su grafici, KPI, tabella e heatmap, e la provenienza delle metriche generate. Queste prove non richiedono una risposta creativa dell'LLM e confrontano input e output deterministici.

6.10.2 Test del workflow

I test del grafo sostituiscono le dipendenze remote con mock e verificano:

- selezione delle sorgenti e parallelismo condizionale;
- convergenza dei risultati nel Conversation Agent;
- separazione della memoria per `thread_id`;
- chiarimenti su richieste incomplete;
- follow-up dal contesto e aggiornamenti selettivi;
- propagazione della risposta finale nello streaming.

6.10.3 Test delle integrazioni

I client MCP e Copernicus sono verificati rispetto a timeout, errori HTTP, payload non validi e tentativi limitati. I test dell'integrazione della dashboard coprono le chiamate HTTP per chat, mutazioni, annullamento, versioni e confronto; quelli degli asset statici verificano i controlli richiudibili e i collegamenti agli endpoint nativi.

La guida tecnica del progetto documenta, al 15 ottobre 2026, 103 test superati nei moduli `test_evals_stats.py`, `test_evals_runner_integrity.py`, `test_evals_validation.py` e `test_evals_infrastructure.py`, e 32 test nella suite `observability/llmops/tests` [25]. Sono esiti di sottoinsiemi della suite, non il conteggio totale corrente né una riesecuzione compiuta per questa tesi. Le prove usano

anche dipendenze simulate e non dimostrano da sole il corretto funzionamento di tutte le sorgenti live.

Per riprodurre tali controlli occorre conservare ambiente, revisione del codice, comando e log dell'esecuzione. Il benchmark del Capitolo 8 aggiunge una misura del workflow su domande di riferimento; la verifica dell'interfaccia renderizzata resta un livello separato.

6.11 Limiti attuali

L'architettura riduce alcuni rischi, ma non elimina i limiti delle fonti e dei modelli:

- GDACS contiene stime che possono essere aggiornate;
- la presenza di una acquisizione STAC non implica che l'immagine sia adatta a valutare un danno;
- il geocoding può essere ambiguo;
- l'LLM può scegliere una formulazione imprecisa, anche se i dati sono strutturati;
- ricerca web e servizi remoti dipendono dalla rete;
- la qualità della risposta non è misurata soltanto dal superamento dei test software.

Il protocollo implementato aggiunge domande con riferimenti, controlli di instradamento e filtri, voti di qualità e statistiche per categoria. Le metriche operative permettono inoltre di localizzare latenze e guasti. Queste evidenze sostengono conclusioni circoscritte al benchmark e ai percorsi osservati; non dimostrano ancora il vantaggio quantitativo del parallelismo o del riuso della memoria, per i quali servono confronti controllati.

Restano utili prove conversazionali con controllo del risultato visualizzato. La qualità in un impiego reale non può essere dedotta dal solo superamento dei test software o dall'assenza di errori tecnici.

Capitolo 7

Osservabilità del workflow e diagnosi

Una risposta corretta, un'esecuzione senza eccezioni e un servizio raggiungibile sono proprietà distinte. Il livello di osservabilità permette di esaminare le ultime due e di ricostruire il percorso che ha prodotto la prima; la valutazione semantica richiede invece i riferimenti e i controlli del Capitolo 8. Questo capitolo descrive la strumentazione LLMOps del prototipo (Prometheus, Grafana, OpenTelemetry/Phoenix) e i suoi limiti operativi, con esempi ancorati alla guida tecnica del progetto [25].

7.1 Log, metriche e trace

I log descrivono eventi puntuali, come un timeout o un nuovo tentativo. Le metriche riassumono quantità numeriche nel tempo, come numero di esecuzioni e distribuzione delle durate. Un trace collega invece le operazioni di una singola richiesta. Ciascuno span rappresenta un intervallo con attributi, eventuale errore e relazione con gli span padre e figli [24].

Queste rappresentazioni rispondono a domande complementari. Un aumento del p95 segnala un gruppo di richieste lente; il trace individua il nodo coinvolto; il log può precisare l'errore remoto. L'assenza di eccezioni non stabilisce che il luogo interpretato sia corretto: una query valida sulla città sbagliata può terminare tecnicamente con

successo.

7.2 Architettura della telemetria

Prometheus legge ogni 15 secondi l'endpoint `/metrics` del backend e conserva le serie temporali. Grafana interroga queste serie con PromQL. OpenTelemetry esporta invece gli span via OTLP/HTTP a Phoenix, che permette di esplorare l'albero della richiesta e le sessioni [2]. Le convenzioni OpenInference caratterizzano span di agenti, modelli, strumenti e retrieval.

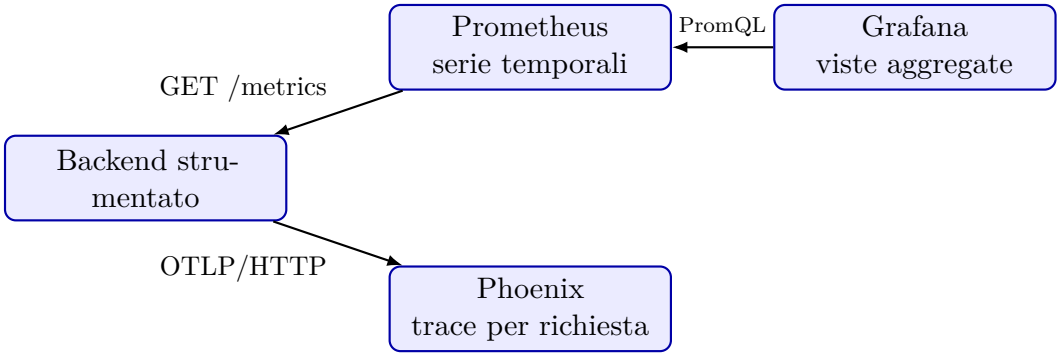


Figura 7.1: Due flussi della telemetria: raccolta delle metriche in modalità pull ed esportazione dei trace in modalità push. Le frecce indicano l'iniziatore della richiesta o dell'invio.

L'overlay `observability/docker-compose.observability.yml` aggiunge i servizi e seleziona l'entrypoint `copernicus_llmops.asgi:app`, che importa e strumenta la stessa applicazione FastAPI. Le impostazioni documentate prevedono volumi persistenti e retention Prometheus di 15 giorni, configurabile. Non sono configurati un Collector OpenTelemetry intermedio, un archivio Loki o un Alertmanager per l'invio delle notifiche. I risultati del Judge restano in JSON e nella propria interfaccia: Phoenix e Prometheus non ne costituiscono l'archivio valutativo.

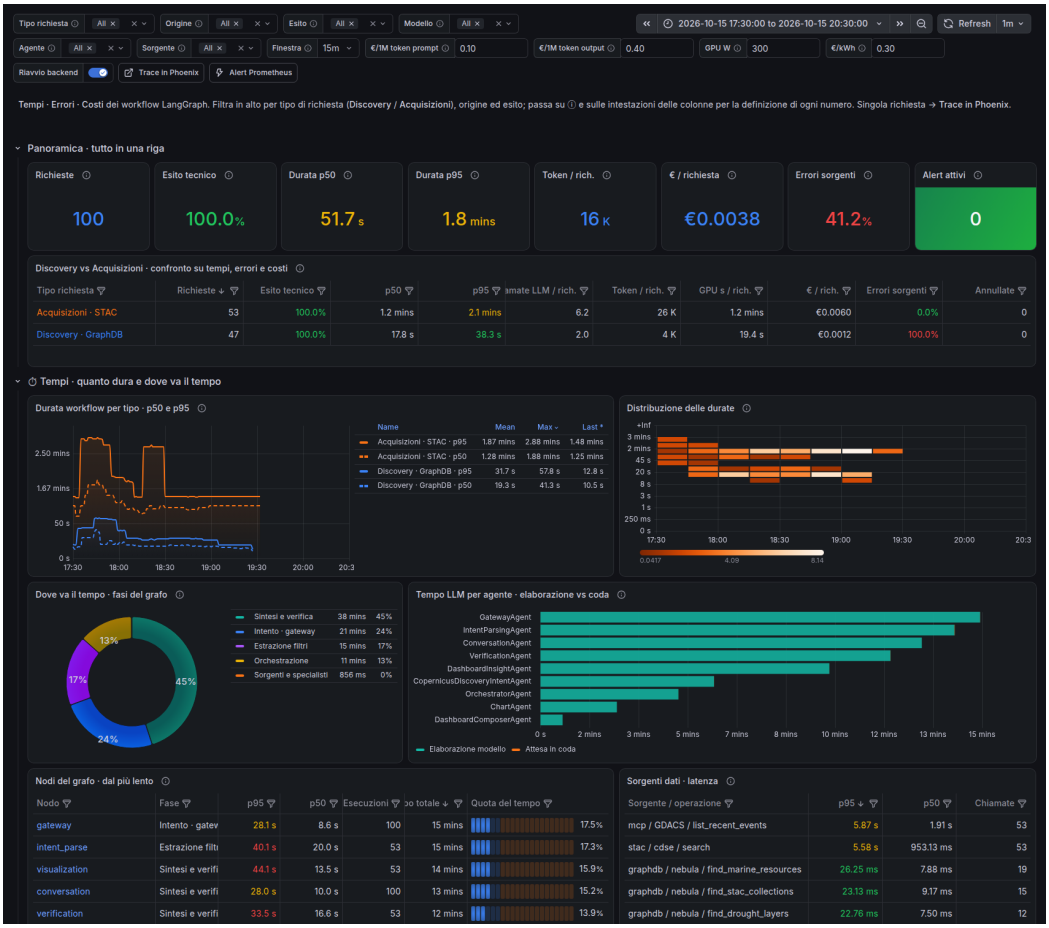


Figura 7.2: Dashboard Grafana *Copernicus · LLMOps*: panoramica di richieste, esiti, durate e confronto Discovery/acquisizioni, con collegamenti a Phoenix e agli alert Prometheus.

7.3 Strumentazione del backend

7.3.1 Bootstrap e wrapper

Il pacchetto sotto `observability/llmops/src/copernicus_llmops/` separa bootstrap, middleware HTTP, metriche, tracing e strumentatori. `instrument_app()` configura l'app prima della compilazione del workflow. I wrapper intercettano le operazioni originali, ne misurano la durata e ne conservano risultato ed errori. L'applicazione della strumentazione è idempotente, per evitare che un secondo bootstrap raddoppi i conteggi.

I nodi vengono avvolti alla registrazione con `StateGraph.add_node()`. Un refactoring di nomi o firme può lasciare un wrapper non applicato: il bootstrap registra avvisi e prosegue con gli altri strumentatori. La disponibilità del backend non garantisce quindi la copertura completa della telemetria. I test del pacchetto verificano anche questi confini con un backend simulato.

7.3.2 HTTP, streaming e correlazione

Il middleware ASGI misura stato e durata della richiesta fino alla conclusione del corpo. Usa il template della route, anziché creare una serie per ogni identificativo di artefatto. Prompt, ID di prodotto e trace ID non diventano etichette Prometheus: una cardinalità elevata moltiplicherebbe inutilmente le serie. Quando viene aperto uno span radice, l'header `X-Trace-ID` permette di ritrovarlo in Phoenix.

Per SSE vengono distinti primo evento, primo contenuto e conclusione dello stream. Un aggiornamento di stato non è ancora una risposta linguistica; un HTTP 200 iniziale può precedere un errore applicativo. Gli esiti separano `completed`, `error`, `client_disconnected` e `interrupted`.

Le route di chat considerate dai pannelli sono `/v1/chat/completions` e `/api/dashboards/{artifact_id}/chat`. Lo stream di avanzamento delle valutazioni `/api/evals/run-stream` è escluso. Un benchmark può quindi produrre workflow e chiamate LLM osservate senza generare campioni di prima risposta della chat.

7.3.3 Workflow, nodi e dipendenze

Gli strumentatori avvolgono i percorsi `WorkflowAgent._invoke()` e `astream()`, registrando durata ed esiti `ok`, `error` o `cancelled`. Il contatore del workflow cresce alla conclusione. Gli span dei nodi riportano input/output sintetici e uno stadio logico: intento, estrazione, orchestrazione, specialista o sintesi. Le eccezioni di controllo del grafo vengono distinte dai guasti ordinari.

I tool MCP sono osservati attraverso `McpToolClient.call_tool()`. Per il grafo vengono misurati i metodi del repository con prefisso `find_`, `list_` o `get_`; le singole chiamate a `NebulaReadClient.execute()` aggiungono span con query e numero di

righe. Una chiamata del repository può eseguire più istruzioni nGQL: il numero di tool GraphDB non coincide necessariamente con il numero di query.

Nel tracing asincrono il contesto viene propagato usando `ContextVar`. Il campo `session.id`, derivato dal thread, raggruppa in Phoenix più richieste della stessa conversazione. Non le trasforma in un unico trace indefinito. Gli span figli includono tempo già compreso nei padri: una durata del workflow di 30 secondi con un figlio di 25 non corrisponde a 55 secondi.

7.3.4 Invocazioni LLM, attesa e token

Per il modello vengono distinti invocazione logica, attesa dello slot di concorrenza e tentativi HTTP. La durata logica comprende coda e retry; tre tentativi per la stessa invocazione producono due retry. Un numero elevato di chiamate in corso può comprendere richieste in coda e non implica altrettante inferenze simultanee.

I conteggi dei token provengono dai campi Ollama `prompt_eval_count` ed `eval_count` del corpo catturato. La velocità di inferenza della singola chiamata è il rapporto fra token generati e `eval_duration`. Il tasso di output registrato per agente in una finestra comprende invece anche i periodi senza attività. Le due quantità possono avere unità token/s e significati diversi.

La somma di `load_duration` e `prompt_eval_duration` fornisce una stima della preparazione al primo token del modello. Poiché la chiamata osservata è senza streaming, non è una misura del primo token ricevuto sul socket. Anche il tempo alla prima risposta della chat è distinto: nel backend descritto il contenuto finale viene consegnato soprattutto alla fine dell'elaborazione, dopo gli aggiornamenti di stato.

Il Judge utilizza un proprio trasporto e resta escluso dalla contabilizzazione LLM di questo pacchetto. Inoltre i tentativi falliti senza corpo valido possono non produrre conteggi di token. La misura esprime il consumo registrato, non una contabilità completa di tutto il calcolo o un costo monetario.

7.4 Interpretazione delle metriche

7.4.1 Contatori, gauge e istogrammi

Un counter descrive un totale cumulativo dall'inizializzazione del processo; un gauge un valore corrente, come le invocazioni in corso. Un istogramma conserva conteggio, somma e bucket cumulativi delle osservazioni. Le stime dei quantili derivano da questi bucket, anziché dall'elenco integrale delle durate [26].

Per stimare gli eventi in un periodo la dashboard usa `increase`; per il ritmo medio al secondo usa `rate`. La prima corregge reset ed estrapola ai margini dell'intervallo: può restituire quantità non intere. Un valore stimato di 14,08 workflow non indica una frazione di esecuzione realmente avvenuta.

Il p50 descrive la mediana; il p95 una soglia sotto cui rientra circa il 95% della distribuzione stimata. Non è il massimo né un indice di qualità. Con pochi campioni o durate oltre il bucket finito più alto, la stima perde forza interpretativa. Il p95 delle durate è inoltre distinto dall'intervallo di confidenza al 95% del benchmark.

7.4.2 Popolazioni e denominatori

La dashboard distingue rapporti con popolazioni differenti:

$$r_{\text{workflow}} = \frac{N_{\text{ok}}}{N_{\text{ok}} + N_{\text{error}}}, \qquad r_{\text{chat}} = \frac{N_{\text{completed}}}{N_{\text{completed}} + N_{\text{error}} + N_{\text{interrupted}}}.$$

Il primo esclude le cancellazioni, il secondo le disconnessioni del client. Queste esclusioni vanno lette insieme alle tabelle degli esiti; non provano che gli abbandoni siano irrilevanti per l'esperienza utente. Se manca un denominatore positivo, il rapporto resta assente. Zero osservato, assenza di attività e assenza di campioni sono condizioni diverse.

Il tempo cumulato dei nodi somma attese e I/O di più esecuzioni. Due nodi da 10 secondi in parallelo possono contribuire 20 secondi cumulati a un workflow di circa 10 secondi. Non si tratta di consumo CPU né di una scomposizione esclusiva della latenza. Anche i p95 di coda e chiamata completa appartengono a distribuzioni diverse e non si sottraggono per ricostruire una chiamata tipica.

7.5 Dashboard Grafana e alert

La configurazione durevole della dashboard è costruita da `observability/grafana/build_dashboard.py`, che genera il JSON con pannelli, query, unità, descrizioni e soglie. Il provisioning carica sorgente Prometheus e dashboard dai file [13]. Una modifica effettuata soltanto nel browser può essere sovrascritta dal provisioning.

La versione documentata comprende 28 pannelli di dati e sei sezioni. Una guida espandibile spiega unità, p50/p95, token, coda ed esiti; un promemoria sui percentili rimane visibile. Le sezioni operative sono riassunte nella Tabella 7.1.

Sezione	Domanda diagnostica e misure
Panoramica	Raccolta recente, alert, esecuzioni concluse, esito tecnico e durate p50/p95.
Diagnosi	Origine degli errori, confronto delle latenze LLM, nodi più lenti e dipendenze MCP/GraphDB.
LLM	Invocazioni attive, campioni, coda, inferenza, caricamenti oltre un secondo, retry e token per agente.
Chat SSE	Stream aperti, completamento, primo evento, prima risposta ed esiti delle route di chat.
HTTP, token e stadi	Freschezza dello scrape, traffico per route, output contabilizzato, tempo cumulato e quota 5xx.

Tabella 7.1: Cinque sezioni di dati, affiancate dalla guida alle metriche.

Il filtro modello riguarda le serie LLM, mentre workflow e dipendenze restano globali. Le curve usano spesso una finestra mobile diversa dall'intero periodo della pagina: un p95 finale della curva e quello della tabella possono differire. Il controllo di raccolta deve precedere la lettura di uno storico.

Le regole Prometheus includono `BackendDown`, errori HTTP, stream falliti, prima risposta lenta, coda Ollama, caricamenti e guasti tool. Una condizione può diventare `pending` e poi `firing` dopo la durata `for`. La finestra su cui si calcola una misura e la persistenza richiesta all'alert sono due intervalli distinti. Senza Alertmanager e destinatari configurati, lo stato firing non implica l'invio di una notifica.

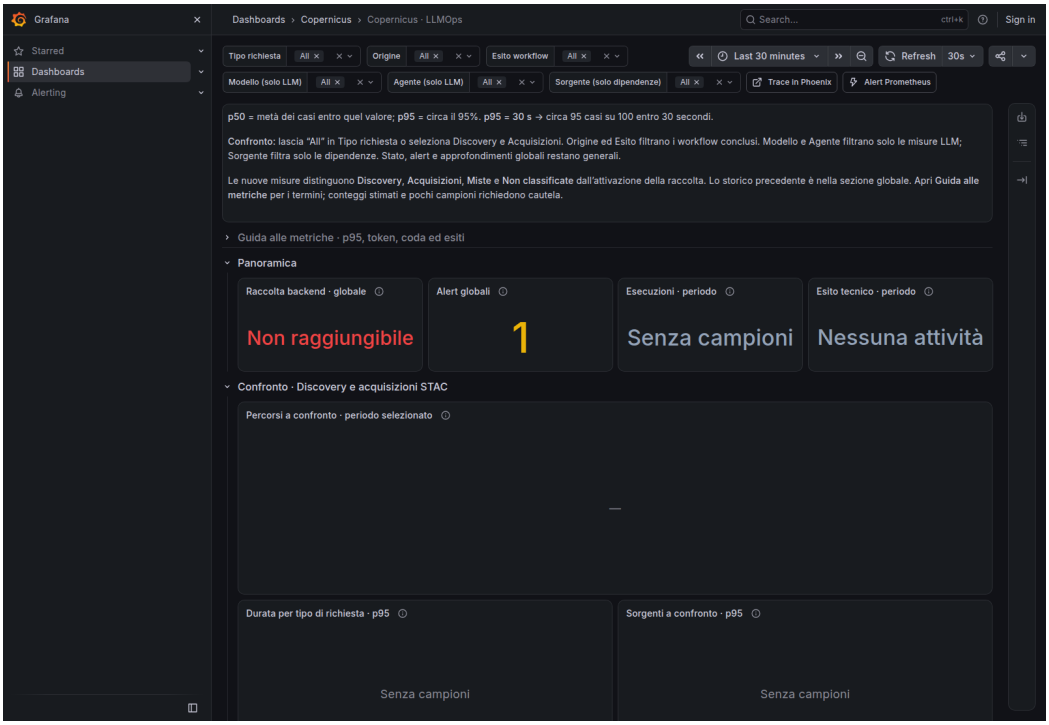


Figura 7.3: Filtri della dashboard LLMOps (tipo richiesta, origine, esito, modello, agente, sorgente) e riga di panoramica con stato della raccolta e alert.

7.6 Diagnosi e limiti della copertura

Per una richiesta lenta occorre controllare raccolta recente e campioni, poi durata del workflow, nodi, coda, retry e dipendenze. Coda alta con inferenza stabile sostiene l'ipotesi di attesa per lo slot; un nodo lento con chiamate LLM brevi orienta verso altri passaggi. Il trace permette di verificare queste ipotesi sul percorso critico, mentre i log precisano gli errori.

Per una risposta errata il confronto riguarda domanda, output del parser, query, record e artefatto. Gli indicatori tecnici possono essere verdi anche quando il luogo è sbagliato. Il benchmark controlla il workflow, ma l'accordo fra risposta, tabella e mappa renderizzate richiede una prova dell'interfaccia.

Non sono misurati direttamente utilizzo GPU/VRAM, risorse di ogni container o accuratezza semantica. I trasporti CDSE STAC, ricerca web e Judge non hanno strumentatori dedicati nel pacchetto descritto: il tempo può apparire nel nodo che li

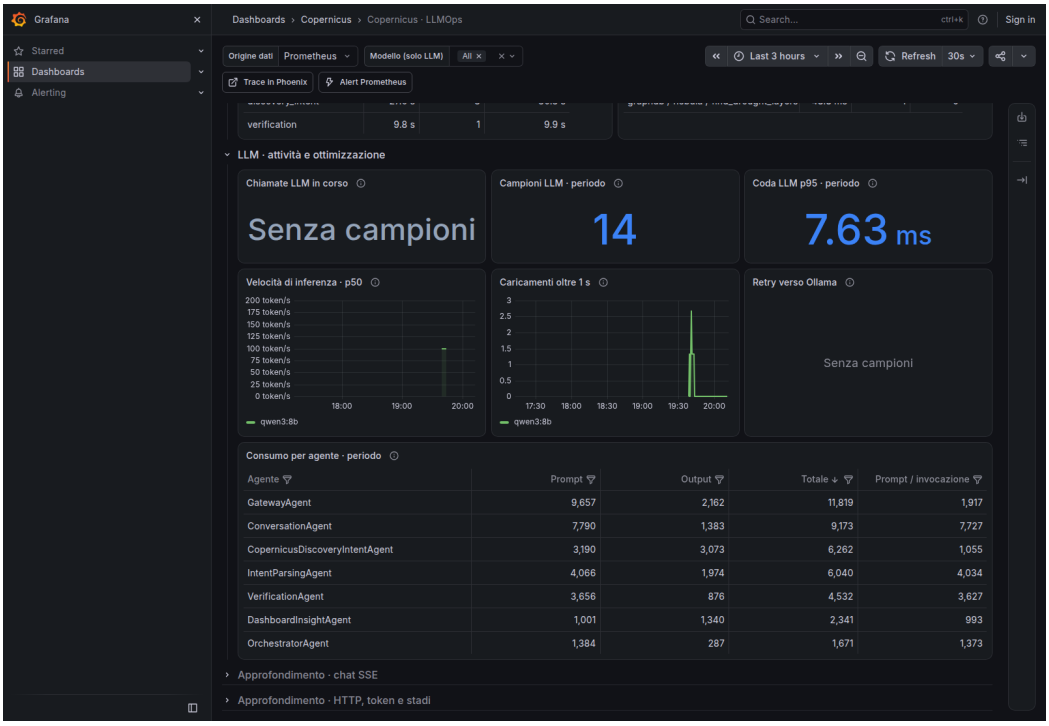


Figura 7.4: Sezione LLM della stessa dashboard: coda, retry, token per agente e salute delle invocazioni al modello.

contiene, senza dettaglio della singola chiamata. Una dipendenza assente dalla tabella dei tool non è per questo inutilizzata.

La cattura di contenuto può registrare prompt, risposte e stato sintetico. Il flag `LLMOPS_CAPTURE_CONTENT` e il limite predefinito di 8.000 caratteri agiscono nei percorsi che li rispettano; filtraggio e troncamento non costituiscono una bonifica universale di ogni attributo. La valutazione di un trace deve considerare anche le evidenze omesse o sintetizzate.

La ricognizione del 15 ottobre 2026 offre un esempio del valore di questi distinguo: Grafana e Prometheus erano disponibili, ma il target backend aveva `up=0` con un errore di risoluzione del nome Docker e `BackendDown` attivo. Grafana conservava comunque dati storici; Phoenix serviva la pagina senza che ciò provasse l'arrivo di nuovi trace. Anche la connessione locale a NebulaGraph era rifiutata [25]. Questa fotografia documentata non è una diagnosi permanente: mostra perché presenza di grafici, salute corrente e correttezza dei risultati debbano essere verificate separatamente.

Capitolo 8

Valutazione con LLM-as-a-Judge

La verifica dei contratti software non esaurisce la valutazione di un agente. Un workflow può rispettare gli schemi e produrre una risposta scorrevole usando un luogo o un periodo sbagliato. Il progetto adotta quindi un benchmark ancorato a evidenze, controlli deterministici e un LLM-as-a-Judge. Questo capitolo descrive il **protocollo 6.0** implementato in `backend/app/evals/` (versione dichiarata in `dataset/schema.py`) e riporta i risultati delle run complete salvate sul dataset corrente. Per confronto storico, in chiusura si richiamano anche le run parziali con protocollo 4.0 documentate nella guida del 15 ottobre 2026 [25]; i due insiemi non sono direttamente somabili perché differiscono per protocollo, hash del dataset e regole di esito.

8.1 Oggetto e perimetro della valutazione

Il sistema sotto esame è il `WorkflowAgent`, comprendente instradamento, interpretazione, recupero e sintesi. Il runner gli sottopone la domanda originale; il Judge valuta risposta ed evidenze osservate. Nella configurazione documentata, il modello dell'agente è `qwen3:8b` e il Judge predefinito è `gemma3:27b`. La famiglia diversa riduce una possibile fonte di preferenza per le proprie risposte, senza garantire imparzialità.

Il Judge non è un modello addestrato nel repository per assegnare voti: viene interrogato con rubriche e output JSON vincolato. È distinto dal nodo Verification dell'agente, che appartiene al percorso di esecuzione. La valutazione a posteriori su casi noti non va confusa con un controllo interno al singolo turno.

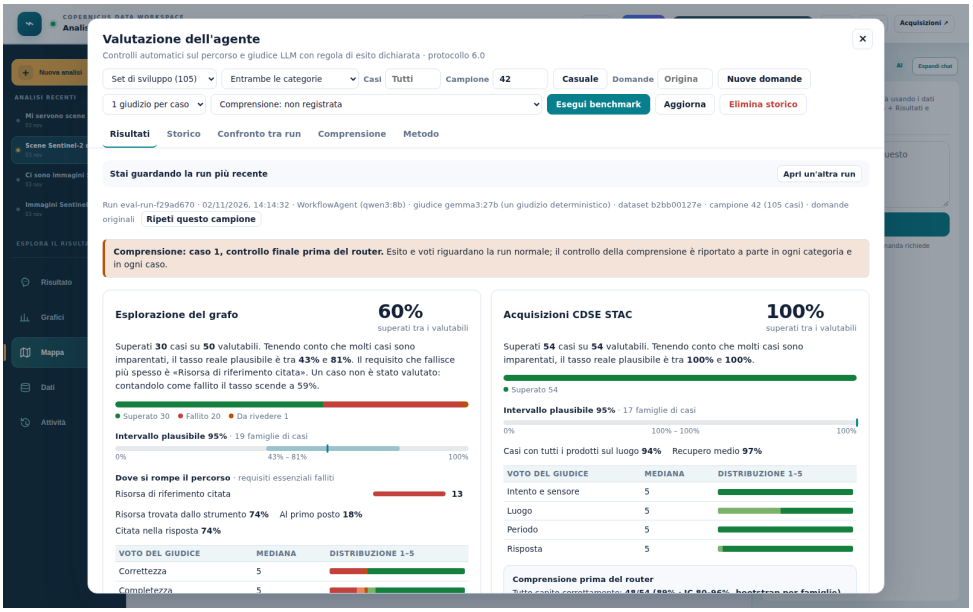


Figura 8.1: Pannello di valutazione nel workspace: esecuzione del benchmark del `WorkflowAgent` (dataset, Judge, controlli e sintesi per categoria).

Opzionalmente, il protocollo 6.0 può registrare una validazione della *comprensione* prima del router, tramite il campo `validation_mode` (`none`/`self_report`/`verifier`/`stage_checks`). L'esito pass/fail del caso resta quello della run ordinaria; le modalità di comprensione sono uno strato aggiuntivo e non sostituiscono i voti del Judge.

La letteratura evidenzia che i giudizi LLM possono risentire di posizione, verbosità e preferenza per le proprie famiglie di modelli [36]: i voti del Judge vanno quindi letti insieme ai controlli deterministici e alle misure di incertezza del protocollo. Inoltre il Judge riceve già l'esito di quei controlli: il voto è *informato* dai fatti verificati e non costituisce una conferma indipendente degli stessi. Non è disponibile, in questa tesi, una validazione umana stratificata dei voti sul compito geospaziale; eventuali falsi pass/fail del Judge restano quindi una minaccia aperta alla validità.

8.2 Dataset ed evidenze di riferimento

8.2.1 Composizione dei casi

Il dataset corrente `backend/app/evals/dataset/benchmark_questions.json` contiene **105 casi**: 51 di categoria `graph` e 54 di categoria `acquisitions`, organizzati rispettivamente in 19 e 17 famiglie (`cluster`). Ogni caso conserva domanda, risposta attesa, strumenti attesi, evidenze, tag e famiglia di scenari.

I casi del grafo coprono Marine, Drought e CDSE/STAC catalogati in NebulaGraph. Le acquisizioni usano collezioni Sentinel-2 L1C/L2A e Sentinel-1 GRD/SLC; alcuni casi prevedono un risultato vuoto: cercare correttamente e dichiarare l'assenza di prodotti può costituire un successo. Esiste inoltre un set perturbato `benchmark_perturbed.json`, riservato a prove di generalizzazione e non usato per tarare le regole.

Questa stratificazione amplia la varietà linguistica ma non rende indipendenti le parafrasi. Domande sullo stesso luogo possono condividere il medesimo errore di geocodifica. La numerosità dei casi deve quindi essere accompagnata da quella delle famiglie effettivamente osservate.

8.2.2 Costruzione della ground truth

Per il grafo, il generatore verifica che titolo, tipo, area, risoluzione e variabili dichiarati nella domanda siano sostenuti dai metadati della risorsa. I riscontri sono conservati in `evidence.grounding`. Un vincolo non supportato impedisce di trattare il caso come riferimento valido.

Per STAC, il riferimento comprende query con bbox, periodo e collection, item osservati e identificativi. I `core_ids` individuano i prodotti che coprono il centro del luogo secondo il criterio del benchmark. La loro copertura misura il recupero di questo insieme di riferimento, senza provare che sia stato recuperato ogni prodotto esistente nel catalogo.

I riferimenti sono snapshot di metadati. Non includono download di raster o misure scientifiche sulle variabili. Il catalogo remoto può cambiare: una differenza di ID rispetto allo snapshot non equivale da sola a un errore dell'agente e va interpretata insieme alla query e ai prodotti osservati.

8.3 Esecuzione del vero workflow

Il runner invoca `WorkflowAgent.run_detailed()` con la domanda originale e un thread nuovo per ogni caso. Per le acquisizioni richiede `include_eval_trace=True`, ottenendo interpretazione, query effettiva e prodotti. L'agente non riceve bbox, sensore o intervallo già risolti dal riferimento: sostituire il workflow con una chiamata diretta STAC misurerebbe soltanto il client, lasciando fuori comprensione e routing.

La sequenza comprende esecuzione dell'agente, controlli deterministici, giudizio, verifica della coerenza dei voti, decisione e salvataggio del JSON della run. `used_tools` deriva dalle sorgenti osservate nel risultato; `expected_tools` resta un requisito del caso. Confondere i due campi renderebbe invisibile un errore di instradamento.

Il timeout dell'agente è configurabile, con default di 600 secondi. Il campo `execut ion_time_ms` comprende anche i passaggi valutativi successivi, incluso il Judge: non rappresenta la sola latenza del workflow in Grafana. La prova non apre il browser; non certifica quindi la corrispondenza fra artefatto e tabella o mappa effettivamente visualizzate.

I moduli principali sono `runner/engine.py`, `runner/checks.py`, `judge/prompts. py`, `judge/evaluator.py` e `stats/metrics.py`, relativi a `backend/app/evals/`. Run di protocolli precedenti (4.0/5.0) non vanno mescolate con le sintesi 6.0 senza dichiarare la versione.

8.4 Controlli deterministici e rubriche ordinali

8.4.1 Controlli sui fatti osservati

I controlli producono esiti vero, falso o non applicabile e una gravità `critical`, `major`, `minor` o `info`. Un fallimento critico impone l'esito negativo. Gli altri sono mostrati al Judge e riportati nel risultato, ma non sono tutti automaticamente bloccanti.

Per il grafo si verifica che il percorso sia Discovery, senza sostituzione con web, GDACS, MCP o STAC operativo. Vengono misurati recupero, posizione e citazione della risorsa di riferimento, oltre ai campi richiesti. La mancata citazione di un target univoco è critica.

Per le acquisizioni, nel protocollo 6.0 sono critici, fra gli altri: percorso STAC corretto; luogo, periodo e sensore della query; filtri richiesti sulla selezione della dashboard; selezione coerente (senza filtri non richiesti aggiunti dall'agente); risposta live da CDSE (un artefatto in cache dopo una risposta vuota è errore); assenza di identificativi inventati. Controlli maggiori riguardano, per esempio, la quota di prodotti sul luogo e la coerenza della sintesi con i prodotti selezionati.

Se una query corretta incontra un guasto della sorgente, il caso può essere non valutabile. Un luogo o periodo già errati non vengono invece giustificati dal guasto successivo.

8.4.2 Dimensioni e soglie

I voti sono interi da 1 a 5 con ancore esplicite per ogni livello. Le rubriche del protocollo 6.0 differiscono per categoria, come mostra la Tabella 8.1.

Categoria	Dimensione	Oggetto del giudizio	Soglia
Grafo	correctness	Correttezza della risorsa e dei vincoli.	4
	completeness	Presenza degli elementi richiesti.	4
	relevance	Focalizzazione e pertinenza.	3
	faithfulness	Assenza di affermazioni false rispetto allo strumento.	4
Acquisizioni	intent	Sensore/prodotto e filtri richiesti.	4
	place	Luogo effettivamente interrogato.	4
	period	Intervallo effettivamente interrogato.	4
	response	Coerenza della sintesi con i prodotti selezionati.	4

Tabella 8.1: Dimensioni ordinali e soglie minime del protocollo 6.0. I controlli critici si applicano indipendentemente dai voti.

Per le acquisizioni, ogni errore sostanziale è attribuito a una sola dimensione (per esempio area troppo ampia → `place`; filtri sbagliati → `intent`). `faithfulness` sul grafo misura solo l'assenza di affermazioni false rispetto all'output dello strumento: una risposta "non ho trovato nulla", quando lo strumento non ha restituito nulla, può essere fedele anche se scorretta rispetto al riferimento.

La media dei quattro voti è un descrittore e non decide l'esito. Una sequenza 5, 5, 5, 1 ha media 4 ma non supera le soglie.

8.4.3 Prompt, parsing e coerenza

Il Judge riceve domanda, riferimento, risposta, traccia e controlli. Nella configurazione ordinaria usa temperatura zero, seed fisso, contesto 16.384 token e output JSON vincolato. Se parsing o voti risultano incoerenti con i fatti verificati, è previsto un ulteriore tentativo; una contraddizione persistente viene marcata per revisione (`review`).

8.5 Stati finali e denominatori

Il protocollo distingue cinque stati:

`pass` controlli critici rispettati e soglie superate;

`fail` fallimento critico o almeno un voto insufficiente;

`review` incoerenza residua dei voti che richiede esame;

`not_evaluable` evidenza non utilizzabile, guasto esterno su richiesta corretta o valutatore inutilizzabile;

`error` errore, timeout o assenza di risposta dell'agente.

Indicando con P, F, R, U, E i conteggi dei cinque stati, i due tassi sono:

$$p_{\text{primario}} = \frac{P}{P+F}, \qquad p_{\text{rigoroso}} = \frac{P}{P+F+R+U+E}. \tag{8.1}$$

Il primo riguarda i casi con esito pass/fail; il secondo include tutti i casi completati in modo conservativo. Se non esistono casi valutabili, il tasso primario è assente. In caso di interruzione vengono salvati i casi completati con `partial=true`. Le sintesi di grafo e acquisizioni restano separate.

8.6 Incertezza e misure di recupero

8.6.1 Intervallo di Wilson

Per $n = P + F$ casi valutabili e proporzione osservata $\hat{p} = P/n$, l'intervallo di Wilson al 95% usa $z \simeq 1{,}96$:

$$\frac{\hat{p} + z^2/(2n) \;\pm\; z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}. \tag{8.2}$$

La procedura è disponibile anche nelle librerie statistiche di riferimento [33]. Wilson tratta i casi come osservazioni indipendenti Bernoulli: non incorpora la dipendenza fra parafrasi della stessa famiglia. Affiancarlo al bootstrap non "risolve" quella dipendenza; i due intervalli rispondono a ipotesi diverse. Un campione piccolo non sostiene una stima precisa; l'intervallo non è la probabilità che la prossima risposta sia corretta.

8.6.2 Bootstrap per famiglie e voti ordinali

Il bootstrap per cluster ricampiona famiglie intere con reinserimento. Si conserva così la dipendenza interna alle famiglie. Con 17 famiglie di acquisizioni e 19 di grafo, la varietà effettiva degli scenari conta quanto il numero di domande. Se tutti i casi passano, il bootstrap può collassare in $[100\%, 100\%]$: è una degenerazione del campione empirico, non una prova di certezza, e va dichiarata affiancando Wilson e numerosità.

I voti 1–5 sono ordinali. Per ogni dimensione il sistema riporta distribuzione, mediana e quartili; l'`overall_score` medio non sostituisce i singoli voti né le regole di esito.

8.6.3 Indicatori indipendenti dai voti

Per il grafo si misurano hit, hit@1, citazione e MRR rispetto alla risorsa di riferimento. Per le acquisizioni si misurano copertura dei `core_ids` e indicatori di recupero sul luogo. Questi indicatori dipendono dalla scelta del riferimento e non misurano tutta la disponibilità globale del catalogo.

8.7 Confronti e ripetizioni

8.7.1 Riproducibilità e stabilità

Il campionamento è mescolato con seed. Con una sola ripetizione del Judge la temperatura è 0; con più ripetizioni (studio di stabilità) è 0,3 e si aggrega con la mediana per dimensione. Le run registrano protocollo, hash del dataset, modello Judge, temperatura, seed e numero di ripetizioni, ma non da sole commit, digest immutabile del modello e stato completo delle sorgenti.

8.7.2 Confronti appaiati fra run

Il confronto ammette run dello *stesso* protocollo, stesso Judge e almeno 20 coppie valutabili per categoria, verificando domanda, evidenza e risposta attesa dei casi comuni. Wilcoxon signed-rank e McNemar (con correzione di Holm sulle dimensioni) sono gli strumenti previsti [31, 18]. La soglia delle 20 coppie non elimina la dipendenza fra famiglie; Wilcoxon richiede inoltre attenzione a pareggi e zeri, frequenti nei voti 1–5. Prima di attribuire un cambiamento a una modifica è utile un confronto A/A della stessa versione.

8.8 Risultati documentati

8.8.1 Run complete con protocollo 6.0

Sul dataset corrente da 105 casi esistono più run complete con protocollo 6.0, Judge `gemma3:27b` e temperatura zero. La Tabella 8.2 riporta la run `eval-run-ca004ba5` (`validation_mode` assente/`none`, seed 40), scelta come riferimento perché è completa, usa il protocollo corrente senza strato aggiuntivo di comprensione e coincide con le metriche di recupero già citate nella guida tecnica. Non è la media di più run né una selezione sul tasso più alto: altre run 6.0 con `self_report`/`verifier`/`stage_checks` restano nello stesso ordine di grandezza e non vanno sommate come osservazioni indipendenti.

Le metriche di recupero indipendenti dai voti, sulla stessa run, sono: hit 71,4%, hit@1 18,4%, citazione 71,4%, MRR 0,295 (49 casi grafo valutabili). Il gap fra hit e

Categoria	Pass (primario)	Pass (rigoroso)	Wilson 95%	Bootstrap 95%
Grafo	31/49 (63,3%)	31/51 (60,8%)	49,3–75,3%	43,8–81,8%
Acquisizioni	54/54 (100%)	54/54 (100%)	93,4–100%	100–100%

Tabella 8.2: Run `eval-run-ca004ba5`. Il tasso primario esclude i due `review` del grafo; il rigoroso li include nel denominatore. Wilson sui casi valutabili non incorpora la dipendenza fra famiglie; il bootstrap al 100% sulle acquisizioni è degenerato (nessun fallimento da ricampionare).

Modalità di fallimento (grafo)	n	Lettura operativa
Target assente dalla lista restituita	8	Recupero/ranking su candidati diversi dal riferimento.
Percorso errato e target non recuperato	4	Instradamento operativo al posto del solo grafo.
Target citato ma voto sotto soglia	4	Controlli critici ok; Judge su completezza/rilevanza.
Target non recuperato (lista vuota)	2	Intent/query senza candidati.
Incoerenza Judge/`review`	2	Citazione corretta ma voti incompatibili con i fatti.

Tabella 8.3: Tassonomia dei 18 `fail` e 2 `review` sul grafo nella run `ca004ba5`, derivata da hit/citazione/percorso nei controlli. Non compaiono casi con target recuperato e non citato: quando il target è in lista e citato, i fallimenti residui sono di qualità o di coerenza del Judge.

hit@1 indica che il riferimento spesso è presente ma non in cima al ranking. Nelle acquisizioni: completezza dei `core_ids` 94,1% e richiamo medio sul nucleo 97,4% (51 casi con traccia di recupero).

Il 100% di pass sulle acquisizioni non implica copertura geografica perfetta. Tre casi passano con avviso maggiore sulla quota di prodotti sul luogo: ad esempio `STAC-017` (Etna, Sentinel-2 L2A) recupera 3 dei 6 `core_ids` (richiamo 0,5) ma tutti i prodotti restituiti sono sul luogo (`on_place_share`=1). Il pass misura quindi il rispetto dei criteri bloccanti del protocollo, non l'esaustività del catalogo sul centro richiesto.

8.8.2 Confronto storico con protocollo 4.0

La guida del 15 ottobre 2026 documenta run con protocollo 4.0 su un dataset da 115 casi (hash diverso). La più ampia, `7dc4eda3`, è parziale (102/115) e riporta 49/51 sul grafo e 32/51 sulle acquisizioni [25]. Il contrasto con la Tabella 8.2 non dimostra da solo un "peggioramento" o un "miglioramento" del sistema: cambiano protocollo, regole critiche, composizione del dataset e completezza della run. Serve soltanto a collocare storicamente quelle cifre e a evitare di presentarle come risultato del protocollo corrente.

8.9 Interpretazione e minacce alla validità

Il contributo valutativo consiste nel rendere ispezionabili domanda, evidenze, percorso, controlli e voti, distinguendo fallimenti dell'agente, problemi esterni e giudizi inutilizzabili. Wilson e bootstrap descrivono incertezza sotto ipotesi dichiarate; non correggono un Judge sistematicamente errato.

La validità interna risente di euristiche dei controlli, riduzione delle evidenze e dipendenze fra casi. La validità esterna rimane limitata da domande curate, numerosità delle famiglie e snapshot. Per rafforzare le conclusioni servono confronti A/A, più scenari indipendenti e verifica dell'interfaccia del workspace, che il benchmark a thread nuovo non osserva.

Capitolo 9

Conclusioni e sviluppi futuri

Il lavoro ha realizzato *Copernicus Data Workspace*: un assistente per consultare fonti geospaziali tramite richieste in linguaggio naturale, esposto in una dashboard nativa autenticata con Keycloak. Il contributo è progettuale e implementativo: responsabilità specializzate e percorsi controllati. Non è stato misurato, a parità di modello e fonti, un vantaggio quantitativo rispetto a un agente singolo o a una pipeline più semplice. Sul piano progettuale sono stati ottenuti, in particolare:

- un workflow LangGraph con confine di dominio, parsing strutturato, percorso Discovery su NebulaGraph, percorso operativo multi-sorgente (web, GDACS, STAC live, CEMS) e percorsi combinati;
- integrazioni MCP per GDACS e CEMS, client STAC diretto per le acquisizioni, harvester e catalogo metadati Marine/STAC/Drought;
- memoria conversazionale Redis, artefatti dashboard verificabili (valori calcolati dal codice, non inventati dal modello), ripresa delle sessioni e interfaccia bilingue IT/EN;
- stack di osservabilità (Prometheus, Grafana, OpenTelemetry/Phoenix) per tempi, errori, nodi e dipendenze;
- un protocollo di valutazione del `WorkflowAgent` (versione 6.0) con controlli deterministici, rubriche ordinali e stima dell'incertezza.

MCP delimita l'integrazione di GDACS e CEMS; CDSE STAC live resta un percorso diretto. Discovery e STAC live restano compiti distinti: il primo sceglie risorse dal catalogo a grafo; il secondo cerca scene sul catalogo remoto.

Risultati della valutazione documentata. Sul dataset corrente da 105 casi, la run completa protocollo 6.0 `eval-run-ca004ba5` (Judge `gemma3:27b`, temperatura 0) riporta sul grafo 31/49 (63,3% primario) e 31/51 (60,8% rigoroso), con hit 71,4% e hit@1 18,4%; sulle acquisizioni 54/54 (100%), con richiamo medio sui `core_ids` 97,4% e alcuni pass che lasciano comunque avvisi maggiori sulla completezza locale. I dettagli, la tassonomia dei fallimenti e i limiti del Judge sono nel Capitolo 8. GDACS, CEMS, follow-up e ripresa sessione restano fuori da questo benchmark. Le cifre storiche del protocollo 4.0 restano un confronto documentale, non il risultato corrente.

Limiti e sviluppi. Restano aperti il miglioramento di recupero e ranking Discovery, una baseline controllata sul contributo multiagente, la validazione umana del Judge e la verifica browser di risposta e artefatto. Sul piano operativo vanno ampliati telemetria, freschezza del catalogo e recupero dai guasti. L'obiettivo è mantenere ogni affermazione proporzionata all'evidenza disponibile.

Bibliografia

[1] Anthropic. Introducing the model context protocol, 2024. Consultata il 8 luglio 2026. https://www.anthropic.com/news/model-context-protocol.

[2] Arize AI. Phoenix: Setup tracing, 2026. Documentazione ufficiale consultata il 15 ottobre 2026. https://arize.com/docs/phoenix/tracing/how-to-tracing/setup-tracing.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020. https://arxiv.org/abs/2005.14165.

[4] Copernicus Data Space Ecosystem. Apis of the copernicus data space ecosystem, 2026. Documentazione ufficiale consultata il 28 luglio 2026. https://dataspace.copernicus.eu/analyse/apis.

[5] Copernicus Data Space Ecosystem. Stac product catalogue, 2026. Documentazione ufficiale consultata il 28 luglio 2026. https://documentation.dataspace.copernicus.eu/APIs/STAC.html.

[6] Copernicus Emergency Management Service. Rapid mapping: Product overview, 2025. Pagina ufficiale aggiornata il 31 gennaio 2025 e consultata il 28 luglio 2026. https://mapping.emergency.copernicus.eu/about/rapid-mapping-manual/product-overview/.

[7] Copernicus Emergency Management Service. Risk and recovery mapping: Service overview, 2025. Pagina ufficiale aggiornata il 29 agosto 2025 e consultata il 28 luglio 2026. https://mapping.emergency.copernicus.eu/about/risk-and-recovery-manual/service-overview/.

[8] Copernicus Emergency Management Service. About copernicus ems, 2026. Pagina ufficiale consultata il 28 luglio 2026. https://emergency.copernicus.eu/about/.

[9] Docker. Docker compose documentation, 2026. https://docs.docker.com/compose/.

[10] Global Disaster Alert and Coordination System. Gdacs feed reference, 2026. Pagina ufficiale consultata il 28 luglio 2026. https://www.gdacs.org/feed_reference.aspx.

[11] Global Disaster Alert and Coordination System. Gdacs knowledge: Scientific overview, 2026. Pagina ufficiale consultata il 28 luglio 2026. https://data.gdacs.org/Knowledge/overview.aspx.

[12] Global Disaster Alert and Coordination System. Gdacs overview, 2026. Pagina ufficiale consultata il 28 luglio 2026. https://www.gdacs.org/About/overview.aspx.

[13] Grafana Labs. Provision Grafana, 2026. Documentazione ufficiale consultata il 15 ottobre 2026. https://grafana.com/docs/grafana/latest/administration/provisioning/.

[14] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, 2012. https://proceedings.neurips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.

[15] LangChain. Langchain documentation, 2026. https://docs.langchain.com/oss/python/langchain/overview.

[16] LangChain. Langgraph documentation, 2026. https://docs.langchain.com/oss/python/langgraph/overview.

[17] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521:436–444, 2015. https://www.nature.com/articles/nature14539.

[18] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[19] Model Context Protocol. Model context protocol specification, 2025. Versione 2025-06-18, consultata il 8 luglio 2026. https://modelcontextprotocol.io/specification/2025-06-18.

[20] NebulaGraph. NebulaGraph Database Manual: Architecture overview, 2026. Documentazione della versione 3.8.0, consultata il 15 ottobre 2026. https://docs.nebula-graph.io/3.8.0/1.introduction/3.nebula-graph-architecture/1.architecture-overview/.

[21] Ollama. Ollama documentation, 2026. https://docs.ollama.com/.

[22] Open WebUI. Open webui documentation, 2026. https://docs.openwebui.com/.

[23] OpenAI. Function calling, 2026. Documentazione ufficiale consultata il 8 luglio 2026. https://developers.openai.com/api/docs/guides/function-calling.

[24] OpenTelemetry Authors. Traces, 2026. Documentazione ufficiale consultata il 15 ottobre 2026. https://opentelemetry.io/docs/concepts/signals/traces/.

[25] Progetto PecilliSpace. Guida completa al progetto: GraphDB, LLM Judge e observability, 2026. Documento tecnico interno, ricognizione del 15 ottobre 2026. File guida-graphdb-judge-observability.md, fornito con i sorgenti della tesi. Contiene descrizione dell'implementazione, risultati salvati e verifiche documentali; i risultati citati non sono nuove esecuzioni.

[26] Prometheus Authors. Histograms and summaries, 2026. Documentazione ufficiale consultata il 15 ottobre 2026. https://prometheus.io/docs/practices/histograms/.

[27] Sebastián Ramírez and contributors. Fastapi documentation, 2026. https://fastapi.tiangolo.com/.

[28] Redis. Redis documentation, 2026. https://redis.io/docs/latest/.

[29] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020. Capitoli su agenti intelligenti. Disponibile anche materiale didattico degli autori: https://people.eecs.berkeley.edu/~russell/aima1e/chapter02.pdf.

[30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. https://arxiv.org/abs/2302.04761.

[31] SciPy Community. Wilcoxon signed-rank test, 2026. Documentazione ufficiale consultata il 15 ottobre 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wilcoxon.html.

[32] SpatioTemporal Asset Catalog. Stac specification, 2025. Specifica ufficiale. https://stacspec.org/en.

[33] statsmodels Developers. Proportion confidence intervals: proportion_confint, 2026. Documentazione ufficiale consultata il 15 ottobre 2026. https://www.statsmodels.org/stable/generated/statsmodels.stats.proportion.proportion_confint.html.

[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. https://arxiv.org/abs/1706.03762.

[35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2210.03629.

[36] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36, 2023. https://arxiv.org/abs/2306.05685.